



UNIVERSITÀ DI PISA

Identificazione e Stima

Stima dello stato di un elicottero 2 DoF mediante EKF ed UKF

Francesco Bonciani 578902

Edoardo Di Scalzo 603117

Luigi Realacci 616067

Anno Accademico 2025/2026

Indice

1	Introduzione	2
2	Implementazione e validazione del modello	2
2.1	Dinamica	3
2.2	Validazione del modello	5
2.3	Sensori	9
2.3.1	Validazione dei Sensori	10
3	Filtraggio	12
3.1	Filtro di Kalman	12
3.2	Filtro di Kalman Esteso	14
3.3	Predizione	15
3.4	Correzione	17
3.4.1	Reiezione delle misure spurie mediante distanza di Mahalanobis	20
4	Filtro di Kalman Unscented	21
4.1	Predizione mediante stato aumentato	21
4.2	Correzione Unscented	25
5	Rauch-Tung-Striebel Smoother	28
6	Analisi dei risultati	30
6.1	Prestazioni dell'EKF	30
6.2	Prestazioni dell'UKF	32
6.3	Confronto tra EKF e UKF	33
6.4	Confronto tra EKF e RTS	34
7	Conclusioni	37
	Appendice	37
A	Confronti delle stime con andamenti reali	38
B	Script MATLAB di inizializzazione	40

1 Introduzione

L'obiettivo del presente progetto è la progettazione e la validazione di algoritmi di stima dello stato applicati ad un sistema dinamico non lineare rappresentato da un elicottero a due gradi di libertà (2DOF). Il sistema considerato descrive i moti di beccheggio ((α)) e imbardata ((β)) di una piattaforma sperimentale azionata da due motori elettrici, il cui comportamento è caratterizzato da accoppiamenti dinamici, attriti viscosi e non linearità trigonometriche. L'attività è stata sviluppata in ambiente MATLAB/Simulink ed ha previsto la realizzazione di un modello fisico del sistema, la simulazione della sensoristica di bordo e l'implementazione di differenti algoritmi di stima basati sul filtro di Kalman. In particolare sono stati sviluppati un Filtro di Kalman Esteso (EKF), un Filtro di Kalman Unscented (UKF) ed uno smoother di Rauch-Tung-Striebel (RTS), con l'obiettivo di confrontarne prestazioni, robustezza numerica e accuratezza nella ricostruzione dello stato. L'architettura di stima utilizza misure provenienti da un accelerometro e da un magnetometro, modellati a partire dalle specifiche di un sensore reale VectorNav VN-100. Particolare attenzione è stata dedicata alla gestione di misure asincrone, alla presenza di outlier e alla validazione statistica delle innovazioni mediante test di bianchezza e distanza di Mahalanobis. I risultati ottenuti consentono di valutare l'efficacia dei diversi approcci di sensor fusion in presenza di dinamiche non lineari e rumori di processo e misura, evidenziandone vantaggi, limiti e possibili sviluppi futuri.

2 Implementazione e validazione del modello

L'elicottero considerato è costituito da una struttura rigida vincolata a ruotare attorno a due assi ortogonali, consentendo i soli moti di pitch e yaw come mostrato in Figura 1.

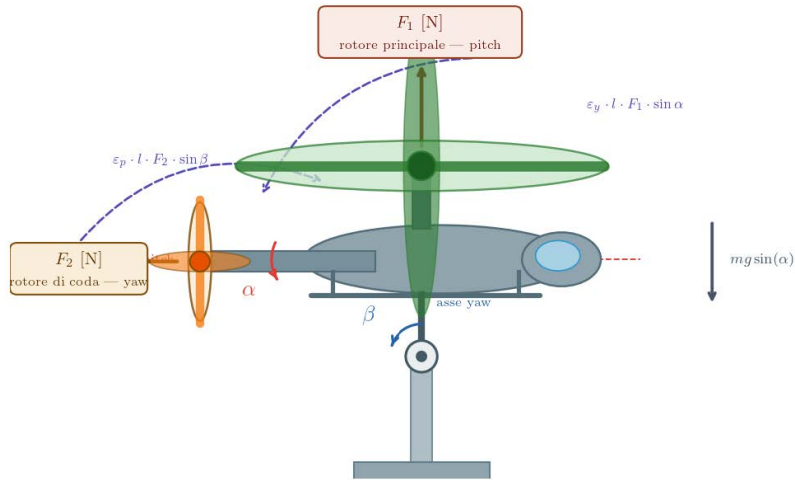


Figura 1: Illustrazione del sistema fisico elicottero 2DOF.

L'implementazione del modello è stata realizzata mediante blocchi MATLAB Function all'interno dell'ambiente Simulink come riportato in Figura 2. Il modello include gli effetti gravitazionali, i contributi dissipativi dovuti agli attriti viscosi e gli accoppiamenti dinamici generati dall'azione simultanea dei due attuatori.

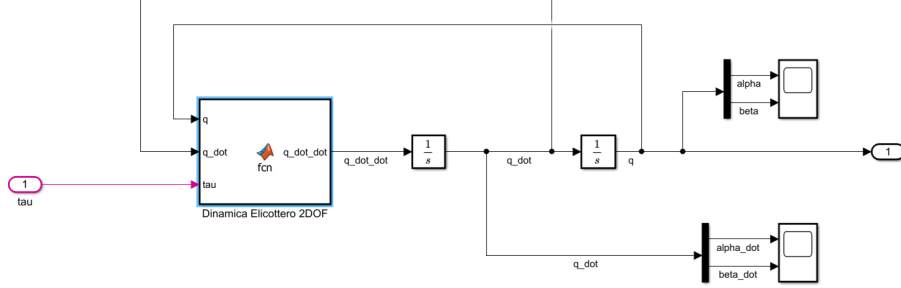


Figura 2: Schema Simulink modello del sistema

2.1 Dinamica

La dinamica dell'elicottero viene descritta attraverso due coordinate generalizzate, corrispondenti agli angoli di pitch (α) e yaw (β). Il modello deriva dall'applicazione delle equazioni di Eulero-Lagrange ad un sistema meccanico rigido soggetto a forze aerodinamiche e contributi dissipativi. L'equazione che descrive il moto di pitch è

$$J_\alpha \ddot{\alpha} = -c_\alpha \dot{\alpha} - mgl \sin(\alpha) + lF_1 \cos(\beta) + \varepsilon_p lF_2 \sin(\beta) \quad (1)$$

dove (J_α) rappresenta il momento di inerzia rispetto all'asse di pitch, (m) è la massa equivalente del sistema, (g) l'accelerazione gravitazionale, (l) il braccio meccanico dell'elicottero e (c_α) il coefficiente di attrito viscoso. L'equazione mostra come il moto di beccheggio sia influenzato da tre contributi principali: la coppia gravitazionale che tende a riportare il sistema verso la posizione di equilibrio, la dissipazione viscosa e la coppia generata dai motori. L'equazione relativa all'asse di yaw assume invece la forma

$$J_\beta(\alpha) \ddot{\beta} = -c_\beta \dot{\beta} + lF_2 \cos(\alpha) + \varepsilon_y lF_1 \sin(\alpha) \quad (2)$$

dove il momento di inerzia equivalente attorno all'asse verticale dipende dall'assetto del sistema secondo la relazione

$$J_\beta(\alpha) = J_y \sin^2(\alpha) + (J_z + ml^2) \cos^2(\alpha) + I_b. \quad (3)$$

Tale dipendenza introduce una significativa non linearità nel modello e rende particolarmente interessante il confronto tra differenti algoritmi di stima. Definendo il vettore di stato

$$x = [\alpha \quad \dot{\alpha} \quad \beta \quad \dot{\beta}]^T \quad (4)$$

si ottiene il modello non lineare nello spazio di stato

$$\dot{x} = f(x, u, w) \quad (5)$$

con vettore di ingresso

$$u = [F_1 \quad F_2]^T \quad (6)$$

Dove F_1 rappresenta la forza del rotore principale ed F_2 la forza del rotore di coda. La funzione MATLAB visibile in Figura 2 implementa le equazioni del modello analizzate in precedenza sotto forma di dinamica inversa secondo l'equazione

$$\ddot{x} = M^{-1}(Bu - n) \quad (7)$$

Dove M è la matrice di inerzia

$$M = \begin{bmatrix} J_\alpha & 0 \\ 0 & J_\beta(\alpha) \end{bmatrix} \quad (8)$$

B è il vettore delle forze generalizzate

$$B = \begin{bmatrix} l \cos(\beta) & \epsilon_p l \sin(\beta) \\ \epsilon_y l \sin(\alpha) & l \cos(\alpha) \end{bmatrix} \quad (9)$$

ed n raccoglie i termini di attrito viscoso e forza di gravità.

$$n = \begin{bmatrix} c_\alpha \dot{\alpha} + mgl \sin(\alpha) \\ c_\beta \dot{\beta} \end{bmatrix} \quad (10)$$

In Tabella 1 sono illustrati i parametri del sistema.

Tabella 1: Specifiche del modello.

Parametro	Simbolo	Valore	Unità di misura
Massa	m	0.2	kg
Braccio delle forze dei rotori	l	0.2	m
Coefficiente attrito viscoso asse beccheggio	c_α	0.01	$\frac{\text{N}\cdot\text{m}\cdot\text{s}}{\text{rad}}$
Coefficiente attrito viscoso asse imbardata	c_β	0.01	$\frac{\text{N}\cdot\text{m}\cdot\text{s}}{\text{rad}}$
Coefficiente di accoppiamento del rotore di coda sull'asse del beccheggio	ϵ_p	0.1	adim.
Coefficiente di accoppiamento del rotore principale sull'asse di imbardata	ϵ_y	0.1	adim.
Accelerazione di gravità	g	9.81	$\frac{\text{m}}{\text{s}^2}$
Momento di inerza rispetto all'asse beccheggio	J_α	0.012	$\text{kg} \cdot \text{m}^2$
Componente di inerzia trasversale	J_y	0.00023	$\text{kg} \cdot \text{m}^2$
Componente di Inerzia verticale	J_z	0.00364	$\text{kg} \cdot \text{m}^2$
Componente inerzia della base	I_b	0.00023	$\text{kg} \cdot \text{m}^2$

2.2 Validazione del modello

Al fine di verificare la correttezza del modello implementato è stata eseguita una campagna preliminare di simulazioni utilizzando differenti profili di ingresso con componenti costanti per i 2 attuatori. Tutti i test sono stati effettuati a partire dalle condizioni iniziali

$$x_0 = [10^\circ \quad 0 \quad 0 \quad 0]^T \quad (11)$$

In una prima fase sono stati applicati ingressi nulli ai due attuatori per osservare la risposta libera del sistema. Questo ha prodotto gli andamenti del vettore di stato visibili nelle Figure 3 e 4.

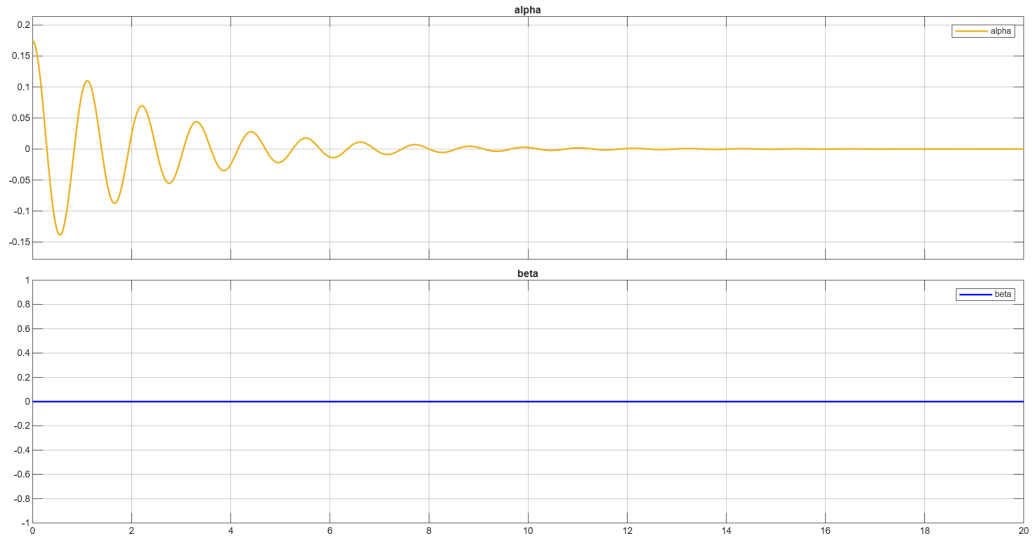


Figura 3: Andamento angoli di beccheggio ed imbardata con ingressi nulli ai rotori

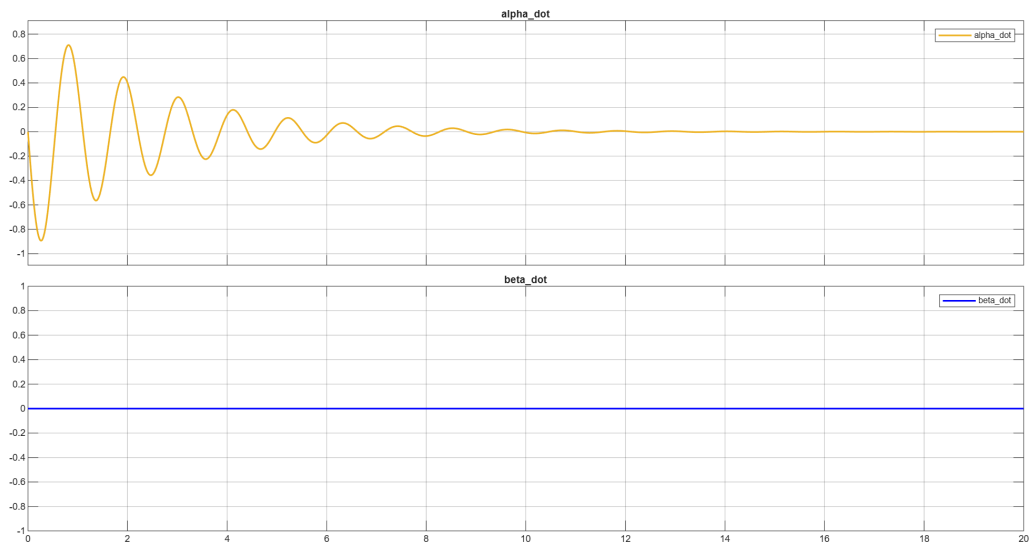


Figura 4: Andamento rate angolari di beccheggio ed imbardata con ingressi nulli ai rotori

Da questi andamenti si verifica la correttezza del modello in quanto l'elicottero cade per effetto della gravità ed oscilla fino a fermarsi intorno al suo equilibrio stabile verticale ($\alpha = 0$).

Successivamente sono stati utilizzati ingressi con componente costante solo su uno dei 2 ingressi per testare i fenomeni di accoppiamento. Ciò ha prodotto gli andamenti visibili nelle Figure 5, 6, 7 ed 8.

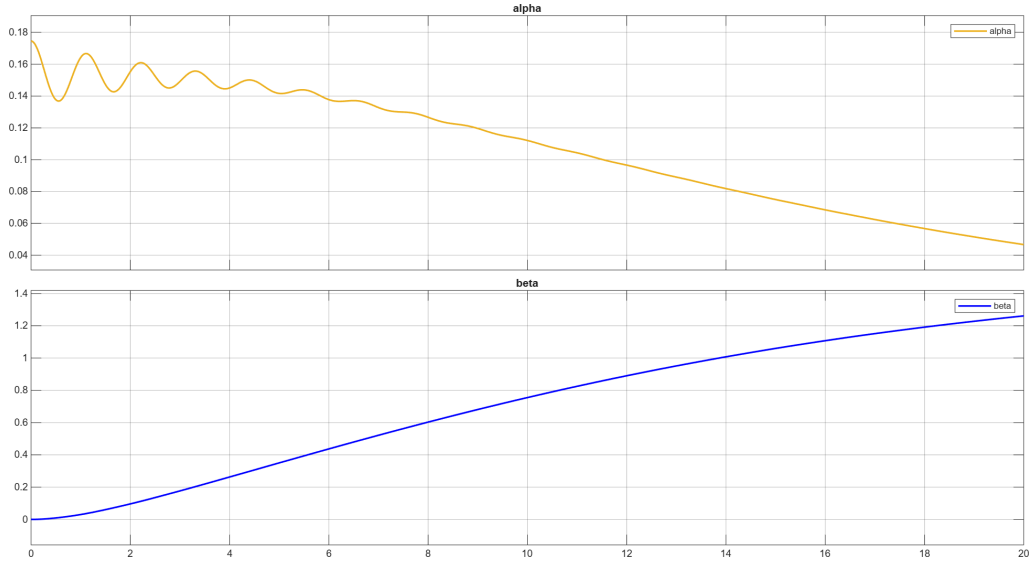


Figura 5: Andamento angoli di beccheggio ed imbardata con ingresso F_1 costante

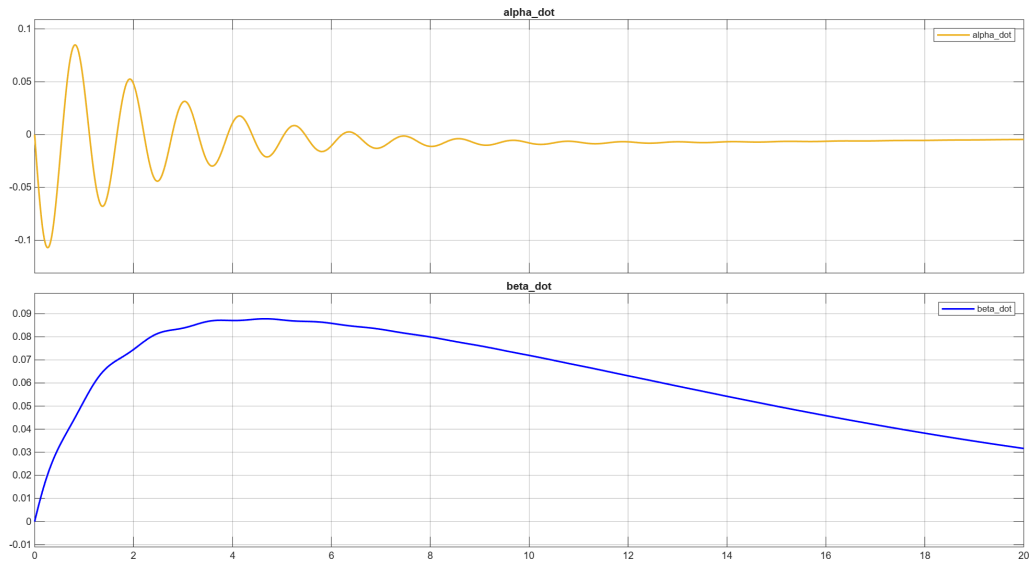


Figura 6: Andamento rate angolari di beccheggio ed imbardata con ingresso F_1 costante

Nel primo caso è visibile la risposta all'azionamento del rotore principale ($F_1 = 0.3$ N). Per quanto riguarda il beccheggio si nota che la forza solleva l'elicottero innescando un transitorio oscillante. Tali oscillazioni vengono smorzate per effetto dell'attrito viscoso. Contestualmente, si osserva un lento abbassamento dell'angolo α : man mano che l'elicottero cambia il suo orientamento la spinta del rotore principale perde di efficacia verticale ($F_1 \cos(\beta)$) confermando l'accoppiamento geometrico del modello. Pur non avendo fornito alcun ingresso al rotore di coda, si osserva che l'elicottero inizia spontaneamente a ruotare su se stesso. Questo comportamento valida l'accoppiamento aerodinamico tra i rotori.

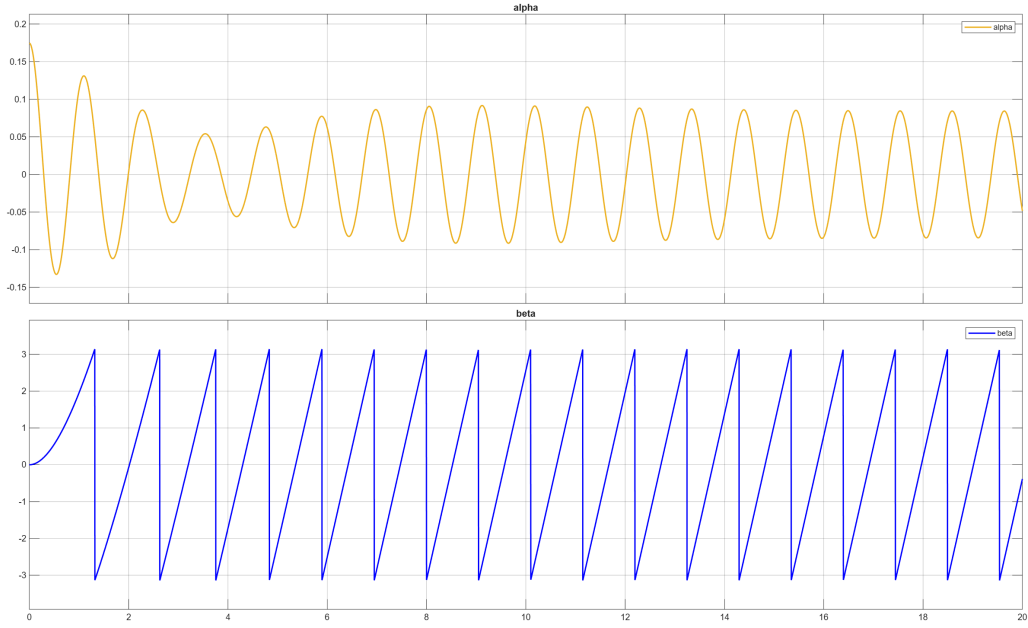


Figura 7: Andamento angoli di beccheggio ed imbardata con ingresso F_2 costante

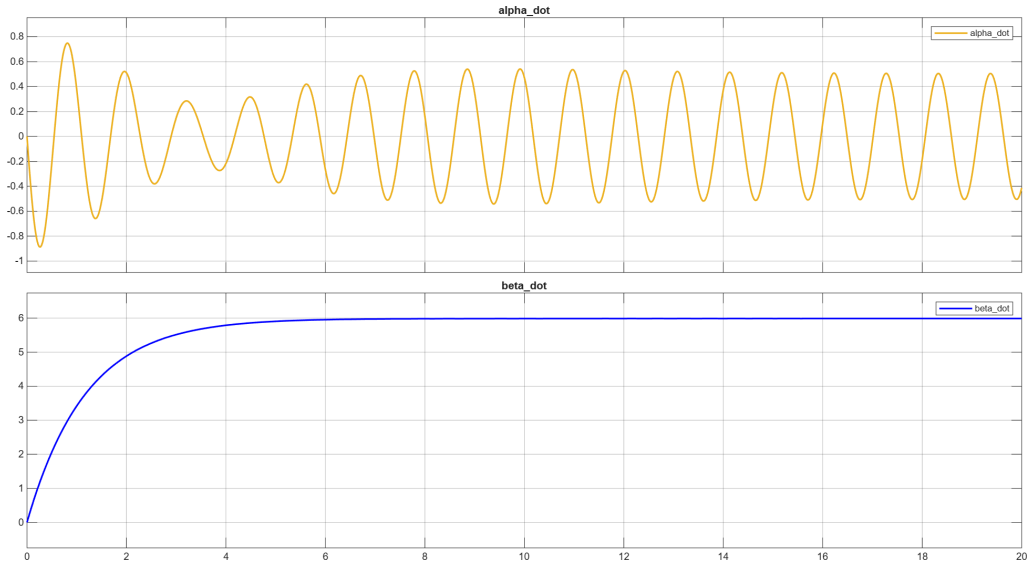


Figura 8: Andamento rate angolari di beccheggio ed imbardata con ingresso F_2 costante

Nel secondo test l'elicottero viene sollecitato accendendo solamente il rotore di coda ($F_2 = 0.3$ N). Essendo privo di un richiamo gravitazionale, l'applicazione di una forza costante genera una rotazione continua sull'asse dell'imbardata. L'elicottero accelera fino a quando la coppia impressa dall'elica di coda viene perfettamente bilanciata dalla resistenza aerodinamica e dall'attrito viscoso. I grafici della simulazione mostrano infatti una velocità angolare che sale e si stabilizza in modo pulito a un valore di crociera costante, confermando la bontà della dinamica del primo ordine implementata per l'orientamento. Pur essendo il rotore principale spento, l'angolo di pitch entra in un'oscillazione continua. Ciò valida l'accoppiamento teorico tra gli assi.

2.3 Sensori

La stima dello stato viene effettuata sfruttando misure provenienti da un accelerometro e da un magnetometro, modellati sulla base delle specifiche del sensore inerziale VectorNav VN-100, visibile in Figura 9.



Figura 9: Sensore inerziale VectorNav VN-100

L'accelerometro viene utilizzato per ricavare una misura indiretta dell'angolo di pitch attraverso la relazione

$$y_{acc} = g \sin(\alpha) + v_{acc}$$

dove (g) rappresenta l'accelerazione gravitazionale e (v_{acc}) è un rumore gaussiano a media nulla. La stima dell'angolo di yaw viene invece ottenuta mediante il magnetometro, il cui modello di osservazione è espresso come

$$y_{magn} = [M_0 \cos(\beta) \quad -M_0 \sin(\beta)]^T + v_{magn}$$

dove ($M_0 = 46.5 \mu\text{T}$) rappresenta il modulo del campo magnetico terrestre locale e (v_{magn}) è il rumore di misura associato. I due sensori operano a frequenze differenti e generano pertanto un problema di fusione dati multi-rate. L'accelerometro fornisce aggiornamenti a frequenza superiore rispetto al magnetometro; di conseguenza i filtri di stima sono stati progettati in modo da eseguire la fase di correzione utilizzando esclusivamente le misure effettivamente disponibili all'istante corrente. In Tabella 2 sono riportate le specifiche per ciascun sensore. Per aumentare il realismo della simulazione sono stati introdotti rumori gaussiani e misure spurie generate casualmente (outlier). Tali anomalie vengono successivamente identificate e rigettate mediante un test basato sulla distanza di Mahalanobis. Nelle Figure 10 e 11 sono riportati gli schemi Simulink implementati per simulare i sensori.

Tabella 2: Specifiche dei sensori.

Sensore	Dev. Standard rumore	Frequenza di lavoro
Accelerometro	$0.0137 \frac{m}{s^2}$	230 Hz
Magnetometro	$0.14 \mu T$	200 Hz

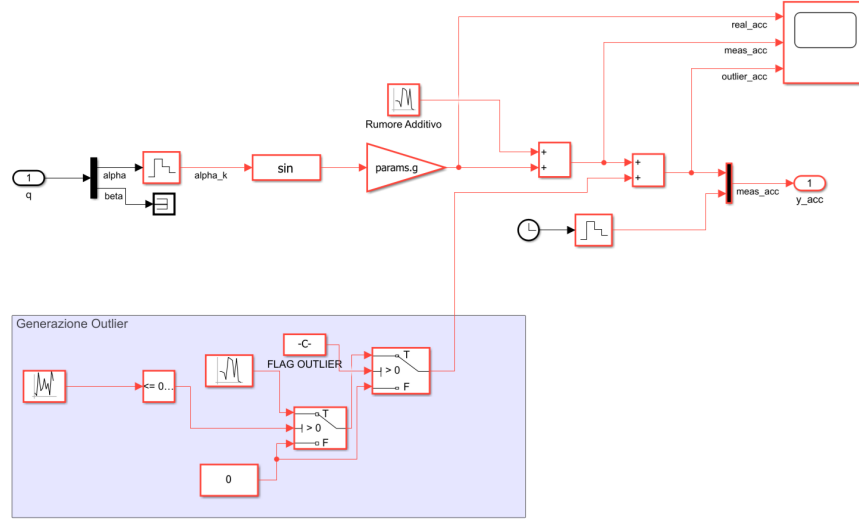


Figura 10: Schema Simulink accelerometro

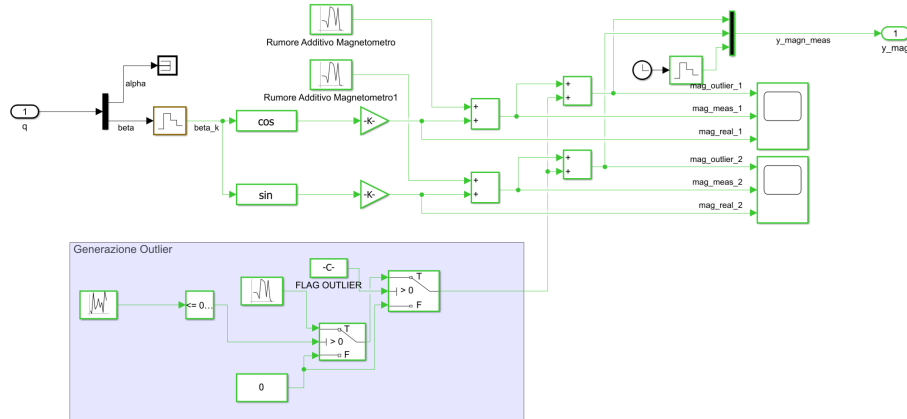


Figura 11: Schema Simulink magnetometro

2.3.1 Validazione dei Sensori

Per validare il modello di osservazione dei sensori è stato eseguito il medesimo test con ingressi nulli illustrato nella sezione 2.2, e sono state monitorate le uscite dei 2 sensori. In Figura 12 è riportato l'andamento delle misure dell'accelerometro con e senza outlier.

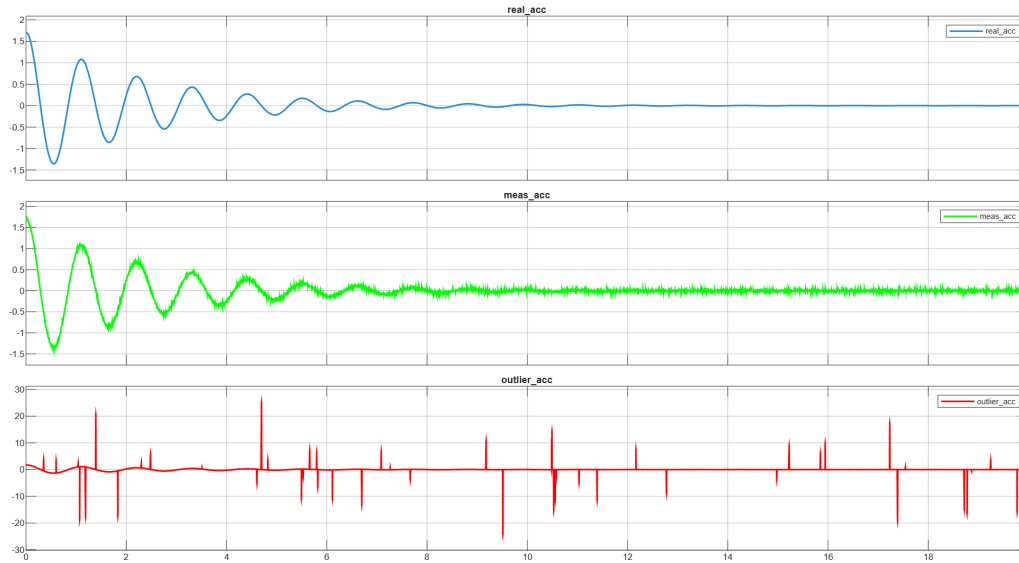


Figura 12: Misura accelerometro con input nullo

Il grafico dell'accelerometro mostra una sinusoide smorzata che converge asintoticamente a zero. L'andamento è perfettamente coerente con l'equazione di osservazione implementata.

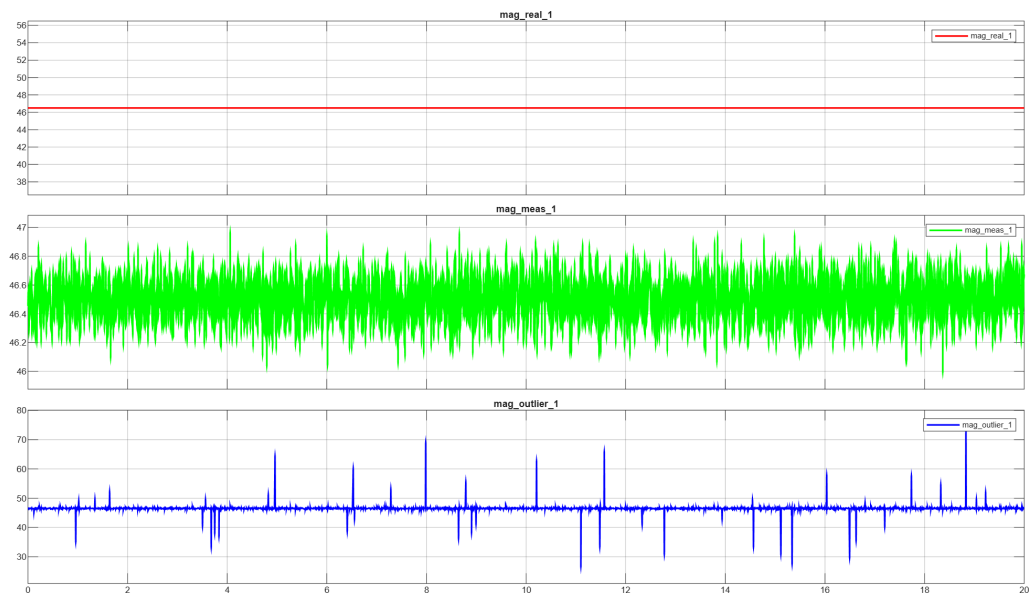


Figura 13: Prima componente della misura di magnetometro con input nullo

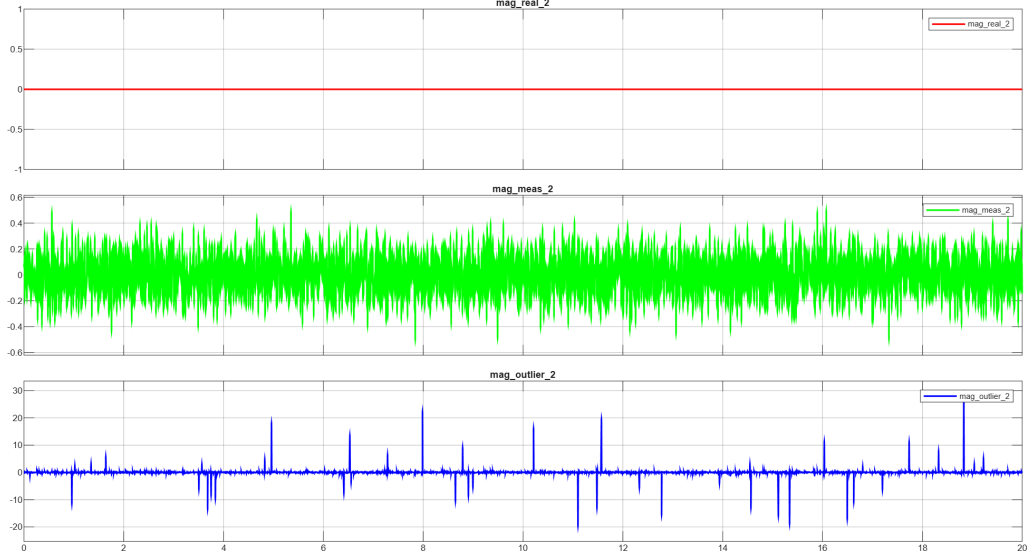


Figura 14: Seconda componente della misura di magnetometro con input nullo

I grafici relativi al magnetometro mostrano segnali che validano il modello matematico in assenza di rotazioni sull'asse verticale. Essendo l'angolo di imbardata costantemente nullo, nelle 2 misure del magnetometro compaiono soltanto i rumori secondo le specifiche del sensore.

3 Filtraggio

3.1 Filtro di Kalman

Il filtro di Kalman serve a stimare lo stato, x , di un sistema minimizzando la varianza dell'errore di stima, P . Nel caso di modello non lineare si usano delle varianti: filtro di Kalman esteso (EKF), filtro di Kalman unscented (UKF) e filtro a particelle (PF). Nel seguente progetto si considerano EKF e UKF con condizioni iniziali per lo stato stimato

$$\hat{x}_0 = \begin{bmatrix} 0.2269 \\ 0.05 \\ 0.0873 \\ -0.05 \end{bmatrix} \quad (12)$$

e con incertezza associata

$$P_0 = \begin{bmatrix} 0.0076 & 0 & 0 & 0 \\ 0 & 0.04 & 0 & 0 \\ 0 & 0 & 0.0195 & 0 \\ 0 & 0 & 0 & 0.04 \end{bmatrix} \quad (13)$$

Le stime fanno riferimento a un ingresso

$$u = \begin{bmatrix} 0.341 + 0.04 \sin(0.6t) + 0.02 \sin(1.4t) \\ -0.006 + 0.010 \sin(0.8t + 0.3) + 0.006 \sin(1.7t) \end{bmatrix} \quad (14)$$

A tale ingresso viene sommato un rumore di attuazione dovuto ai motori principali, il cui modello scelto è EMAX MT2204, visibile in Figura 15.



Figura 15: Modello del motore EMAX MT2204

Le traiettorie del vettore di stato prodotte eccitando il sistema con l'ingresso definito sono visibili in Figura 17.

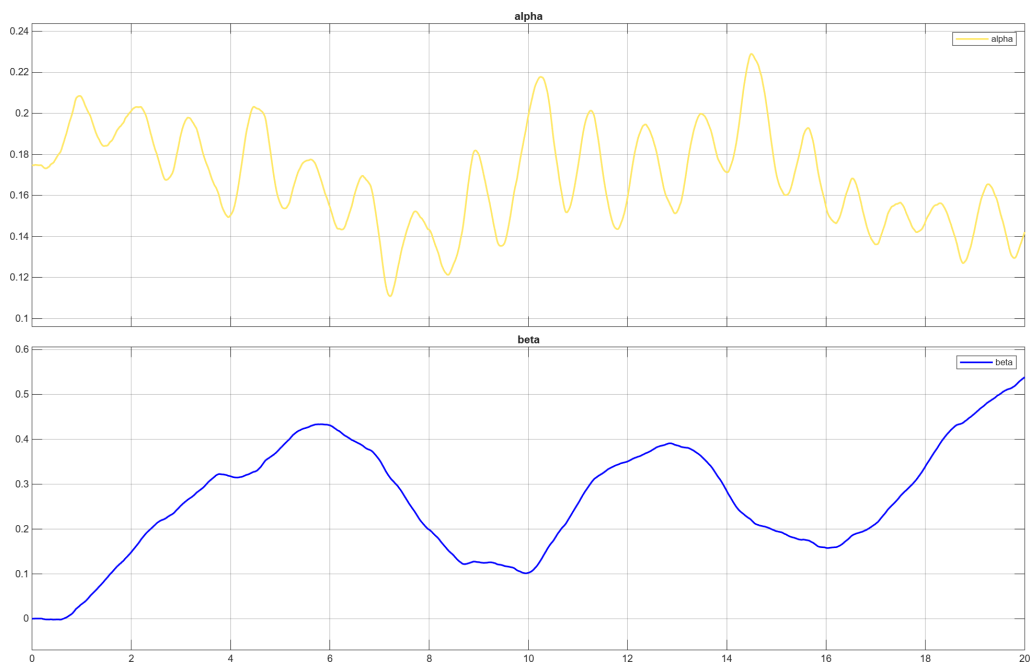


Figura 16: Angoli di beccheggio e imbardata relativi agli ingressi sinusoidali.

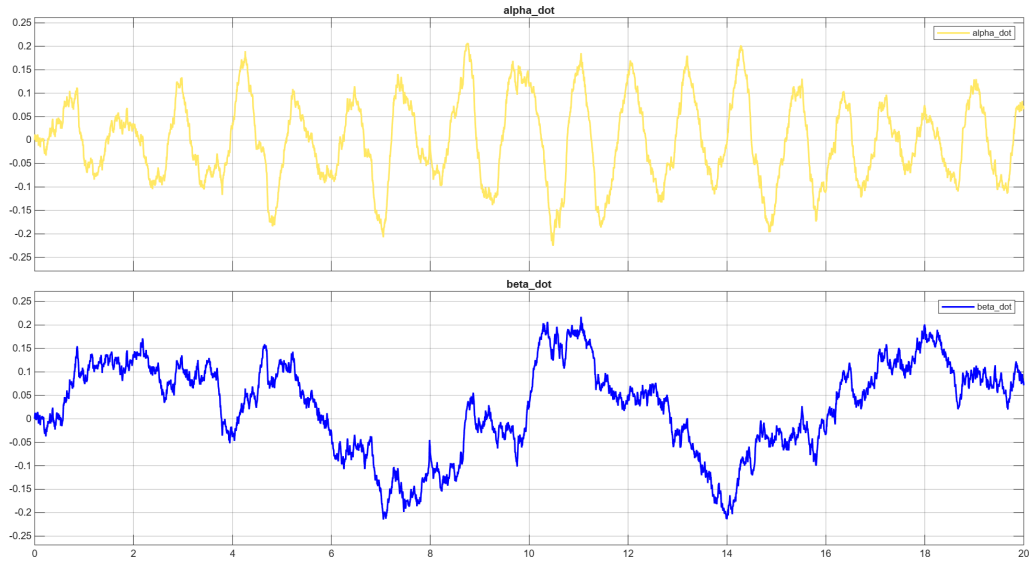


Figura 17: Rate angolari di beccheggio e imbardata relativi agli ingressi sinusoidali.

3.2 Filtro di Kalman Esteso

Il filtro di Kalman esteso (EKF) rappresenta una generalizzazione del filtro di Kalman lineare ai sistemi non lineari. L'idea alla base dell'algoritmo consiste nel linearizzare localmente le equazioni del sistema mediante uno sviluppo di Taylor al primo ordine attorno alla stima corrente dello stato. Nel caso in esame il modello dinamico dell'elicottero presenta non linearità dovute ai termini trigonometrici presenti nelle equazioni del moto e nella funzione di osservazione. Per tale motivo l'utilizzo del filtro di Kalman classico non risulta applicabile direttamente. L'algoritmo implementato è costituito da due fasi principali:

1. Predizione dello stato e della covarianza tramite il modello dinamico.
2. Correzione mediante le misure provenienti dai sensori.

Poiché accelerometro e magnetometro operano a frequenze differenti, l'EKF è stato implementato secondo una logica multi-rate. La fase di predizione viene eseguita con il tempo di campionamento del sensore più veloce, mentre la fase di aggiornamento utilizza solamente le misure effettivamente disponibili all'istante corrente. Di conseguenza la dimensione del vettore delle osservazioni e delle matrici associate alla correzione varia nel tempo. La dimensione della matrice di covarianza dello stato rimane invece costante ed è pari a (4×4) , essendo determinata esclusivamente dalla dimensione del vettore di stato. L'EKF è stato implementato in ambiente MATLAB/Simulink, come visibile in Figura 18.

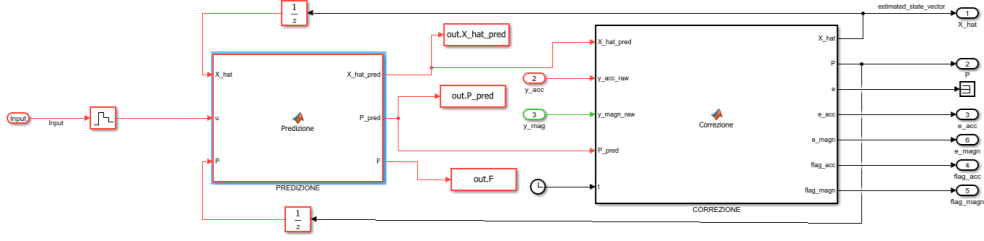


Figura 18: Schema Simulink del filtro di Kalman esteso.

3.3 Predizione

La fase di predizione utilizza il modello dinamico non lineare discretizzato mediante integrazione di Eulero. La stima dello stato all'istante successivo viene ottenuta mediante la relazione

$$\hat{x}_{k+1|k} = \hat{x}_{k|k} + T_s f(\hat{x}_{k|k}, u_k) \quad (15)$$

dove T_s rappresenta il tempo di campionamento fondamentale del sistema. Per descrivere l'incertezza associata agli attuatori viene introdotto un rumore di processo additivo agente sugli ingressi

$$u_k = \bar{u}_k + w_k \quad (16)$$

con

$$w_k \sim \mathcal{N}(0, Q). \quad (17)$$

La propagazione della covarianza richiede la linearizzazione locale del modello mediante il calcolo della matrice Jacobiana

$$F_k = \left. \frac{\partial f(x, u)}{\partial x} \right|_{\hat{x}_{k|k}} \quad (18)$$

e della matrice di ingresso del rumore

$$D_k = \left. \frac{\partial f(x, u)}{\partial w} \right|_{\hat{x}_{k|k}}. \quad (19)$$

La covarianza predetta viene quindi aggiornata secondo

$$P_{k+1|k} = F_k P_{k|k} F_k^T + D_k Q D_k^T. \quad (20)$$

Il codice della MATLAB Function che implementa la fase di predizione è ispezionabile nel listato seguente:


```

1 function [X_hat_pred, P_pred, F] = Predizione(X_hat, u, P, params)
2     % PREDIZIONE EKF - Modello con Rumore di Attuazione sui Motori
3
4     %% 1. Lettura dello Stato e Ingressi
5     alpha      = X_hat(1);
6     alpha_dot  = X_hat(2);
7     beta       = X_hat(3);
8     beta_dot   = X_hat(4);
9
10    F1 = u(1);
11    F2 = u(2);
12
13    %% 2. Estrazione Parametri Fisici
14    m = params.m;      l = params.l;      g = params.g;
15    Ja = params.J_alpha; Jy = params.J_y;    Jz = params.J_z;
16    Ib = params.I_b;    ca = params.c_alpha; cb = params.c_beta;
17    ep = params.epsilon_p; ey = params.epsilon_y;
18    dt = params.st;
19
20    %% 3. Matrice di Covarianza del Rumore di attuazione sugli ingressi (Q)
21    % Q e' 2x2. Contiene la varianza del rumore dei motori EMAX MT2204
22    % params.sigma_F = 0.09 [Newton]
23    Q = [params.sigma_F^2, 0;
24         0,                params.sigma_F^2];
25
26    %% 4. MODELLO DINAMICO NON LINEARE
27    J_beta = Jy * sin(alpha)^2 + (Jz + m*l^2) * cos(alpha)^2 + Ib;
28
29    alpha_ddot = (-ca * alpha_dot - m * g * l * sin(alpha) + ...
30                 l * F1 * cos(beta) + ep * l * F2 * sin(beta)) / Ja;
31
32    beta_ddot = (-cb * beta_dot + l * F2 * cos(alpha) + ...
33                ey * l * F1 * sin(alpha)) / J_beta;
34
35    %% 5. PREDIZIONE DELLO STATO (Eulero)
36    X_hat_pred = [alpha      + dt * alpha_dot;
37                 alpha_dot + dt * alpha_ddot;
38                 beta       + dt * beta_dot;
39                 beta_dot   + dt * beta_ddot];
40
41    %% 6. CALCOLO JACOBIANA DI PROCESSO (A) -> F = I + A*dt
42    % Derivate per la riga 2 (alpha_ddot)
43    A21 = (-m * g * l * cos(alpha)) / Ja;
44    A22 = -ca / Ja;
45    A23 = (-l * F1 * sin(beta) + ep * l * F2 * cos(beta)) / Ja;
46    A24 = 0;
47
48    % Derivate per la riga 4 (beta_ddot)
49    N = -cb * beta_dot + l * F2 * cos(alpha) + ey * l * F1 * sin(alpha);
50    D_den = J_beta;
51    dN_dalpha = -l * F2 * sin(alpha) + ey * l * F1 * cos(alpha);
52    dD_dalpha = 2 * Jy * sin(alpha) * cos(alpha) - 2 * (Jz + m*l^2) * cos(alpha)
    * sin(alpha);

```

```

53
54 A41 = (D_den * dN_dalpha - N * dD_dalpha) / (D_den^2);
55 A42 = 0;
56 A43 = 0;
57 A44 = -cb / D_den;
58
59 % Assemblaggio e discretizzazione
60 A = [ 0, 1, 0, 0;
61       A21, A22, A23, A24;
62       0, 0, 0, 1;
63       A41, A42, A43, A44];
64
65 F = eye(4) + A * dt;
66
67 %% 7. CALCOLO JACOBIANA DEI DISTURBI (D)
68 % Rappresenta l'impatto di 1 Newton di errore del motore sulle accelerazioni.
69 % La matrice e' 4x2 perche' mappa 2 rumori su 4 stati.
70
71 d_alpha_ddot_w1 = (l * cos(beta)) / Ja;
72 d_alpha_ddot_w2 = (ep * l * sin(beta)) / Ja;
73
74 d_beta_ddot_w1 = (ey * l * sin(alpha)) / J_beta;
75 d_beta_ddot_w2 = (l * cos(alpha)) / J_beta;
76
77 D = [0, 0;
78       d_alpha_ddot_w1 * dt, d_alpha_ddot_w2 * dt;
79       0, 0;
80       d_beta_ddot_w1 * dt, d_beta_ddot_w2 * dt];
81
82 %% 8. PREDIZIONE DELLA COVARIANZA
83 % Diffusione dell'incertezza dei motori negli stati tramite la matrice D
84 P_pred = F * P * F' + D * Q * D';
85
86 end

```

Listing 1: Blocco di predizione dell'EKF

3.4 Correzione

La fase di correzione sfrutta le misure provenienti dall'accelerometro e dal magnetometro per aggiornare la stima predetta. Il modello di osservazione adottato è

$$y_k = h(x_k) + v_k \quad (21)$$

dove v_k rappresenta un rumore gaussiano bianco caratterizzato da covarianza R . L'innovazione viene definita come

$$e_k = y_k - h(\hat{x}_{k|k-1}) \quad (22)$$

e rappresenta la discrepanza tra misura reale e misura prevista dal modello. Linearizzando la funzione di osservazione si ottiene la matrice Jacobiana

$$H_k = \left. \frac{\partial h(x)}{\partial x} \right|_{\hat{x}_{k|k-1}}. \quad (23)$$

La covarianza dell'innovazione risulta

$$S_k = H_k P_{k|k-1} H_k^T + R_k \quad (24)$$

mentre il guadagno di Kalman è dato da

$$K_k = P_{k|k-1} H_k^T S_k^{-1}. \quad (25)$$

La stima corretta dello stato viene infine ottenuta tramite

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k e_k. \quad (26)$$

Per garantire una migliore stabilità numerica la matrice di covarianza aggiornata è stata calcolata utilizzando la forma di Joseph, che preserva la simmetria e la definita positività della matrice P . Il codice della MATLAB Function che implementa la fase di correzione è ispezionabile nel listato seguente:

```

1 function [X_hat, P, e, e_acc,e_magn, flag_acc,flag_magn] = Correzione(X_hat_pred,
    y_acc_raw, y_magn_raw, P_pred, params, t, MAHALANOBIS, M_0, m_threshold)
2     % FUNZIONE DI CORREZIONE EKF
3     % Modello di osservazione:
4     % y_acc = gsin(alpha_k) + v_alpha_k
5     % y_magn = [M_0*cos(beta_k), -M_0*sin(beta_k)] + [v_beta_k, v_beta_k]
6
7     g = params.g;
8     dt = params.st;
9
10    % Inizializzazione
11    X_hat = X_hat_pred;
12    P = P_pred;
13
14    % Input (x_k/k-1)
15    alpha = X_hat_pred(1);
16    beta = X_hat_pred(3);
17
18    % Uscite Sensori
19    t_acc = y_acc_raw(2);
20    t_magn = y_magn_raw(3);
21    y_acc = y_acc_raw(1);
22    y_magn = y_magn_raw(1:2);
23
24
25    % Covarianze errore di misura

```

```

26 R = diag([params.R_acc, params.R_magn, params.R_magn]);
27
28 % Matrice M = dh_k/dv_k valutata in x_k/k-1
29 M = eye(3);
30
31 % Matrice H = dh_k/x_k valutata in x_k/k-1
32 H = [g*cos(alpha), 0, 0, 0; ...
33      0, 0, -M_0*sin(beta), 0; ...
34      0, 0, -M_0*cos(beta), 0];
35
36 % Innovazione
37 e = [y_acc; y_magn] - [g*sin(alpha); M_0*cos(beta); -M_0*sin(beta)];
38
39 % Calcolo delle singole distanze di mahalanobis
40 H_acc = H(1, :);
41 e_acc = e(1);
42 M_acc = M(1, 1);
43 R_acc = R(1, 1);
44 S_acc = H_acc * P_pred * H_acc' + M_acc * R_acc * M_acc';
45 mahalanobis_acc = sqrt(e_acc' / S_acc * e_acc);
46
47 H_magn = H(2:3, :);
48 e_magn = e(2:3);
49 M_magn = M(2:3, 2:3);
50 R_magn = R(2:3, 2:3);
51 S_magn = H_magn * P_pred * H_magn' + M_magn * R_magn * M_magn';
52 mahalanobis_magn = sqrt(e_magn' / S_magn * e_magn);
53
54 % Verifica della disponibilita' delle misure
55 if MAHALANOBIS
56     flag_acc = (mahalanobis_acc <= m_treshold) && (t_acc <= t) && (t - dt <
57 t_acc);
58     flag_magn = (mahalanobis_magn <= m_treshold) && (t_magn <= t) && (t - dt
59 < t_magn);
60 else
61     flag_acc = (t_acc <= t) && (t - dt < t_acc);
62     flag_magn = (t_magn <= t) && (t - dt < t_magn);
63 end
64
65 % Costruzione delle matrici in base alle misure valide
66 % Entrambe le misure valide
67 if flag_acc && flag_magn
68     H_curr = H;
69     e_curr = e;
70     M_curr = M;
71     R_curr = R;
72
73 elseif flag_acc % Solo misura accelerometro disponibile
74     H_curr = H_acc;
75     e_curr = e_acc;
76     M_curr = M_acc;
77     R_curr = R_acc;
78
79 elseif flag_magn % Solo misura magnetometro disponibile

```

```

78     H_curr = H_magn;
79     e_curr = e_magn;
80     M_curr = M_magn;
81     R_curr = R_magn;
82
83     else % Nessuna misura disponibile
84         return;
85     end
86
87     % Covarianza dell'innovazione
88     S = H_curr * P_pred * H_curr' + M_curr * R_curr * M_curr';
89
90     % Guadagno di Correzione
91     L = P_pred * H_curr' / S;
92
93     % Aggiornamento stato e covarianza
94     X_hat = X_hat_pred + L * e_curr;
95
96     X_hat(1) = mod(X_hat(1) + pi, 2*pi) - pi; % Wrap di alpha
97     X_hat(3) = mod(X_hat(3) + pi, 2*pi) - pi; % Wrap di beta
98
99     % Forma di Joseph
100    P = (eye(4) - L * H_curr) * P_pred * (eye(4) - L * H_curr)' + L * R_curr * L';
101
102 end

```

Listing 2: Blocco di correzione dell'EKF

3.4.1 Reiezione delle misure spurie mediante distanza di Mahalanobis

Durante il funzionamento reale di un sistema di stima possono verificarsi misure anomale dovute a disturbi elettromagnetici, errori di acquisizione o malfunzionamenti temporanei dei sensori. L'utilizzo diretto di tali misure nella fase di correzione può causare errori significativi nella stima dello stato o addirittura portare alla divergenza del filtro. Per limitare tale fenomeno è stata implementata una logica di validazione statistica basata sulla distanza di Mahalanobis. Tale grandezza misura quanto una determinata innovazione risulti compatibile con la distribuzione statistica prevista dal filtro. Nel caso generale la distanza di Mahalanobis è definita come

$$D_M = \sqrt{e_k^T S_k^{-1} e_k} \quad (27)$$

dove (e_k) rappresenta il vettore di innovazione e (S_k) la relativa matrice di covarianza. Nel progetto sviluppato si è scelto di effettuare un controllo separato per accelerometro e magnetometro. Tale approccio permette di rigettare selettivamente il sensore affetto da un'anomalia senza compromettere l'utilizzo delle informazioni provenienti dagli altri dispositivi. Qualora la distanza di Mahalanobis superi il valore di soglia, la misura viene considerata non affidabile e viene esclusa dall'aggiornamento del filtro. In tal modo si ottiene un significativo incremento della robustezza dell'algoritmo in presenza di misure spurie.

4 Filtro di Kalman Unscented

Sebbene l'EKF rappresenti uno degli strumenti più diffusi per la stima di sistemi non lineari, le sue prestazioni dipendono fortemente dalla qualità della linearizzazione locale. In presenza di dinamiche fortemente non lineari la propagazione degli errori ottenuta mediante lo sviluppo di Taylor può risultare inaccurata. Per superare tale limitazione è stato implementato un filtro di Kalman Unscented (UKF). Questo approccio evita completamente il calcolo delle matrici Jacobiane e utilizza invece la Trasformata Unscented per propagare media e covarianza attraverso il modello non lineare. L'idea fondamentale consiste nel rappresentare la distribuzione di probabilità dello stato mediante un insieme deterministico di punti, detti Sigma Points, opportunamente scelti affinché abbiano la stessa media e covarianza della distribuzione originale. Tali punti vengono propagati attraverso il modello non lineare e successivamente ricombinati per ottenere una stima accurata delle statistiche dello stato. L'approccio Unscented risulta particolarmente adatto al caso dell'elicottero 2DOF, caratterizzato dalla presenza di non linearità trigonometriche e da una matrice di inerzia dipendente dall'assetto. Anche l'UKF è stato implementato in ambiente MATLAB/Simulink, come visibile in Figura 19.

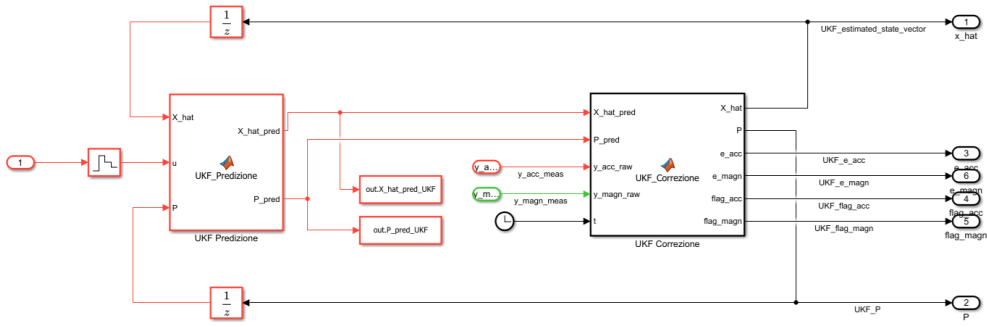


Figura 19: Schema Simulink del filtro di Kalman Unscented.

4.1 Predizione mediante stato aumentato

Nel caso in esame il rumore di processo non agisce direttamente sulle variabili di stato, ma sugli ingressi di controllo associati ai due motori. Per descrivere correttamente tale fenomeno è stata adottata una formulazione a stato aumentato. Il vettore di stato esteso assume quindi la forma

$$x_a = \begin{bmatrix} x \\ w \end{bmatrix} = \begin{bmatrix} \alpha \\ \dot{\alpha} \\ \beta \\ \dot{\beta} \\ w_{F_1} \\ w_{F_2} \end{bmatrix} \quad (28)$$

dove w_{F_1} e w_{F_2} rappresentano i rumori associati ai due attuatori. Ne consegue che nel caso della predizione $n = 6$ pari alla dimensione del vettore di stato aumentato. La covarianza aumentata risulta

$$P_a = \begin{bmatrix} P_{k|k} & 0 \\ 0 & Q_k \end{bmatrix}. \quad (29)$$

A partire da tale matrice vengono generati $2n + 1$ Sigma Points secondo la relazione

$$x_i = \begin{cases} \hat{x}_{k|k} + \sqrt{n + \lambda} \gamma_i & i > 0 \\ \hat{x}_{k|k} & i = 0 \\ \hat{x}_{k|k} - \sqrt{n + \lambda} \gamma_i & i < 0 \end{cases} \quad (30)$$

dove γ_i rappresenta la i -esima colonna della matrice delle covarianze aumentata, fattorizzata mediante SVD. Ogni Sigma Point viene successivamente propagato attraverso il modello dinamico non lineare dell'elicottero ottenendo una nuvola di punti che approssima l'evoluzione della distribuzione di probabilità dello stato. La nuova media e la nuova covarianza vengono infine calcolate come combinazione pesata dei Sigma Points propagati, con pesi definiti secondo la trasformata unscented scalata. Per la covarianza i pesi sono:

$$w_i = \begin{cases} \frac{\lambda}{n + \lambda} + 1 - \alpha^2 + \beta & i = 0 \\ \frac{1}{2(n + \lambda)} & i \neq 0 \end{cases} \quad (31)$$

Per la media i pesi sono:

$$w_i = \begin{cases} \frac{\lambda}{n + \lambda} & i = 0 \\ \frac{1}{2(n + \lambda)} & i \neq 0 \end{cases} \quad (32)$$

Con

$$\lambda = \alpha^2(n + \kappa) - n \quad (33)$$

Scegliendo come valori:

$$\begin{cases} \alpha = 0.01 \\ \beta = 2 \\ \kappa = 0 \end{cases} \quad (34)$$

La trasformata unscented implementata nella fase di predizione è:

$$[\hat{x}_{k+1|k}, P_{k+1|k}] = UT([\hat{x}_{k|k} \quad 0]^T, \text{diag}(P_{k|k}, Q_k), f_k) \quad (35)$$

Il codice della MATLAB Function che implementa la fase di predizione dell'UKF è ispezionabile nel listato seguente:

```

1 function [X_hat_pred, P_pred] = UKF_Predizione(X_hat,u, params, P)
2
3 lambda = params.lambda;
4 n = params.n;
5
6 n_state = params.n_state;
7
8 % Estrazione parametri
9
10 m = params.m;    J_a = params.J_alpha;    l = params.l;
11 g = params.g;    c_a = params.c_alpha;    c_b = params.c_beta;
12 e_p = params.epsilon_p;    e_y = params.epsilon_y;
13
14 F_1 = u(1);
15 F_2 = u(2);
16
17 % Inizializzazione vettore sigma_points
18 sigma_points = zeros(n, 2*n + 1);
19
20 Q = diag([params.sigma_F^2, params.sigma_F^2]);
21
22 P_a = blkdiag(P, Q);
23
24 % Definizione stato aumentato
25 X_a = [X_hat; zeros(params.n_input, 1)];
26
27 % 1. Decomposizione ai Valori Singolari (SVD)
28 [U, S, ~] = svd(P_a);
29
30 % 2. Protezione Numerica Assoluta:
31 % Trasforma eventuali autovalori microscopici negativi in zero
32 S_positiva = max(S, 0);
33
34 % 3. Calcolo della Radice di Matrice
35 GAMMA = U * sqrt(S_positiva);
36
37 % Estrazione Sigma-points(da -n ad n)
38 for i =-n:n
39     idx = i + n + 1;
40     if i == 0
41         sigma_points(:, idx) = X_a;    % Sigma-point centrale
42     elseif i > 0
43         sigma_points(:, idx) = X_a + sqrt(n + lambda) * GAMMA(:, i);
44     elseif i < 0
45         sigma_points(:, idx) = X_a - sqrt(n + lambda) * GAMMA(:, abs(i));
46     end
47 end
48
49
50 %-----
51 % PROPAGAZIONE SIGMA-POINTS
52 %-----
53

```



```

54 sigma_points_pred = zeros(n_state, 2*n + 1);
55
56 dt = params.st;
57
58 for i = 1:(2*n + 1)
59
60     % Estrazione Stato
61     alpha = sigma_points(1, i);
62     alpha_dot = sigma_points(2, i);
63     beta = sigma_points(3, i);
64     beta_dot = sigma_points(4, i);
65
66     % Estrazione rumore di attuazione
67     w_1 = sigma_points(5, i);
68     w_2 = sigma_points(6, i);
69
70     F_1_noise = F_1 + w_1;
71     F_2_noise = F_2 + w_2;
72
73
74     % Dinamica del sistema
75     J_beta = params.J_y * sin(alpha)^2 + (params.J_z + params.m * params.l^2) *
        cos(alpha)^2 + params.I_b;
76
77     alpha_dot_dot = 1/J_a * (-c_a * alpha_dot - m* g * l * sin(alpha) + l *
        F_1_noise * cos(beta) + e_p * l * F_2_noise * sin(beta));
78     beta_dot_dot = 1/J_beta * (-c_b * beta_dot + l * F_2_noise * cos(alpha) + e_y
        * l * F_1_noise * sin(alpha));
79
80
81     % Eulero:
82     sigma_points_pred(1, i) = alpha + dt * alpha_dot;
83     sigma_points_pred(2, i) = alpha_dot + dt * alpha_dot_dot;
84     sigma_points_pred(3, i) = beta + dt * beta_dot;
85     sigma_points_pred(4, i) = beta_dot + dt * beta_dot_dot;
86
87 end
88
89
90
91 % Predizione
92 X_hat_pred = zeros(4, 1);
93 for i = 1:(2*n + 1)
94     X_hat_pred = X_hat_pred + params.Wm(i) * sigma_points_pred(:, i);
95 end
96
97 P_pred = zeros(4, 4);
98
99 for i = 1:(2*n + 1)
100     P_pred = P_pred + params.Wc(i) * (sigma_points_pred(:, i) - X_hat_pred) * (
        sigma_points_pred(:, i) - X_hat_pred)';
101 end
102
103 end

```

4.2 Correzione Unscented

La fase di aggiornamento utilizza il medesimo principio della Trasformata Unscented applicato al modello di osservazione. In questo caso, si sfrutta l'ipotesi di rumore di misura puramente additivo per efficientare la fase di correzione: ciò permette di evitare l'aumento del vettore di stato, mantenendo la dimensione del problema strettamente legata agli stati fisici ($n = 4$).

Applicando la Trasformata Unscented alle stime a priori si ottengono la media prevista delle misure $\hat{y}_{k|k-1}$, la covarianza delle osservazioni P_{yy} e la covarianza incrociata stato-misura P_{xy} :

$$[\hat{y}_{k|k-1}, P_{yy}, P_{xy}] = \text{UT}(\hat{x}_{k|k-1}, P_{k|k-1}, h_k) \quad (36)$$

Poiché il rumore di misura è additivo, l'incertezza intrinseca dei sensori, rappresentata dalla matrice di covarianza R , viene sommata direttamente a P_{yy} per ottenere la covarianza dell'innovazione S :

$$S = P_{yy} + R \quad (37)$$

Il guadagno di Kalman Unscented viene quindi calcolato rispetto all'incertezza totale S :

$$K_k = P_{xy} S^{-1} \quad (38)$$

Tale guadagno viene infine utilizzato per aggiornare lo stato e la covarianza secondo:

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k(y_k - \hat{y}_{k|k-1}) \quad (39)$$

$$P_{k|k} = P_{k|k-1} - K_k S K_k^T \quad (40)$$

Il codice della MATLAB Function che implementa la fase di correzione è ispezionabile nel listato seguente:

```

1 function [X_hat, P, e_acc,e_magn,flag_acc, flag_magn] = UKF_Correzione(X_hat_pred
    , P_pred, y_acc_raw, y_magn_raw, params, M_0, MAHALANOBIS, m_threshold, t)
2 % CORREZIONE UKF, conviene sfruttare il fatto che il rumore di misura e'
3 % additivo (in questo caso non si avra' lo stato aumentato con i rumori di
4 % misura)
5
6 n_state = params.n_state;
7 lambda = params.lambda_c;
8 g = params.g;
9 dt = params.st;
10
11 % Pre-allocazione (se nessuna misura e' disponibile, rimango con la
12 % stima predetta)

```

```

13 P = P_pred;
14 X_hat = X_hat_pred;
15
16 % Uscite sensori
17 y_acc = y_acc_raw(1);
18 t_acc = y_acc_raw(2);
19
20 y_magn = y_magn_raw(1:2);
21 t_magn = y_magn_raw(3);
22
23 % Inizializzazione vettore sigma_points
24 sigma_points = zeros(n_state, 2*n_state + 1);
25
26 % 1. Decomposizione ai Valori Singolari (SVD)
27 [U, S, ~] = svd((P_pred + P_pred')/2);
28
29 % 2. Protezione Numerica Assoluta:
30 % Trasforma eventuali autovalori microscopici negativi in zero
31 S_positiva = max(S, 0);
32
33 % 3. Calcolo della Radice di Matrice
34 GAMMA = U * sqrt(S_positiva);
35
36 % Estrazione Sigma-points(da -n_state ad n_state)
37 for i = -n_state:n_state
38     idx = i + n_state + 1;
39     if i == 0
40         sigma_points(:, idx) = X_hat_pred;           % Sigma-point centrale
41     elseif i > 0
42         sigma_points(:, idx) = X_hat_pred + sqrt(n_state + lambda) * GAMMA(:, i);
43     elseif i < 0
44         sigma_points(:, idx) = X_hat_pred - sqrt(n_state + lambda) * GAMMA(:, abs
45         (i));
46     end
47 end
48
49 %-----
50 % PROPAGAZIONE ATTRAVERSO I SENSORI (TRASFORMATO UNSCENTED)
51 %-----
52 Y_points = zeros(3, 2*n_state + 1);
53 for i = 1:(2*n_state + 1)
54     alpha = sigma_points(1, i);
55     beta = sigma_points(3, i);
56
57     % Modello di osservazione
58     Y_points(1, i) = g * sin(alpha);
59     Y_points(2, i) = M_0 * cos(beta);
60     Y_points(3, i) = -M_0 * sin(beta);
61
62 end
63
64 % Media
65 y_hat = zeros(3, 1);

```

```

66 for i = 1:(2*n_state + 1)
67     y_hat = y_hat + params.Wm_c(i) * Y_points(:, i);
68 end
69
70 % Covarianza
71 P_yy = zeros(3, 3);
72 P_xy = zeros(n_state, 3);
73 for i = 1:(2*n_state + 1)
74     P_yy = P_yy + params.Wc_c(i) * ((Y_points(:, i) - y_hat) * (Y_points(:, i) -
75     y_hat)');
76     P_xy = P_xy + params.Wc_c(i) * ((sigma_points(:, i) - X_hat_pred) * (Y_points
77    (:, i) - y_hat)');
78 end
79
80 %-----
81 % Calcolo delle distanze di mahalanobis per i singoli sensori
82 %-----
83 e = [y_acc; y_magn] - y_hat;
84
85 R = diag([params.R_acc, params.R_magn, params.R_magn]);
86
87 % Accelerometro
88 e_acc = e(1);
89 R_acc = R(1, 1);
90 S_acc = P_yy(1, 1) + R_acc; % Sfrutto rumore additivo
91 mahalanobis_acc = sqrt(e_acc' / S_acc * e_acc);
92
93 % Magnetometro
94 e_magn = e(2:3);
95 R_magn = R(2:3, 2:3);
96 S_magn = P_yy(2:3, 2:3) + R_magn;
97 mahalanobis_magn = sqrt(e_magn' / S_magn * e_magn);
98
99 % Verifica della disponibilita' delle misure
100 if MAHALANOBIS
101     flag_acc = (mahalanobis_acc <= m_treshold) && (t_acc <= t) && (t - dt < t_acc
102     );
103     flag_magn = (mahalanobis_magn <= m_treshold) && (t_magn <= t) && (t - dt <
104     t_magn);
105 else
106     flag_acc = (t_acc <= t) && (t - dt < t_acc);
107     flag_magn = (t_magn <= t) && (t - dt < t_magn);
108 end
109
110 % Costruzione delle matrici in base alle misure valide
111 % Entrambe le misure valide
112 if flag_acc && flag_magn
113     e_curr = e;
114     S_curr = P_yy + R;
115     P_xy_curr = P_xy;

```

```

116
117 elseif flag_acc      % Solo misura accelerometro disponibile
118     e_curr = e_acc;
119     S_curr = S_acc;
120     P_xy_curr = P_xy(:, 1);
121
122 elseif flag_magn      % Solo misura magnetometro disponibile
123     e_curr = e_magn;
124     S_curr = S_magn;
125     P_xy_curr = P_xy(:, 2:3);
126
127 else                  % Nessuna misura disponibile
128     return;
129 end
130
131 % CORREZIONE
132 % Guadagno di correzione
133 L = P_xy_curr / S_curr;
134
135 %  $x_k/k$ 
136 X_hat = X_hat_pred + L * e_curr;
137
138 X_hat(1) = mod(X_hat(1) + pi, 2*pi) - pi; % Wrap di alpha
139 X_hat(3) = mod(X_hat(3) + pi, 2*pi) - pi; % Wrap di beta
140
141 %  $P_k/k$ 
142 P = P_pred - L * S_curr * L';

```

Listing 4: Blocco di correzione dell'UKF

5 Rauch-Tung-Striebel Smoother

Lo smoother Rauch-Tung-Striebel (RTS) rappresenta un'estensione del filtro di Kalman che consente di migliorare la stima degli stati passati sfruttando informazioni disponibili in istanti successivi. A differenza di EKF e UKF, che operano in tempo reale utilizzando esclusivamente misure presenti e passate, il filtro RTS è un algoritmo offline che richiede la disponibilità dell'intera sequenza temporale delle osservazioni. L'algoritmo è composto da due fasi:

1. filtraggio in avanti;
2. regolarizzazione all'indietro.

Nella prima fase viene eseguito il filtro di Kalman producendo, per ogni istante temporale, le stime corrette $\hat{x}_{k|k}$ e le relative covarianze $P_{k|k}$. Successivamente viene effettuata una scansione inversa della traiettoria a partire dall'ultimo campione disponibile. Definendo

$$C_k = P_{k|k} F_{k+1}^T P_{k+1|k}^{-1} \quad (41)$$

si ottiene il guadagno di smoothing utilizzato per correggere le stime passate. La stima regolarizzata assume quindi la forma

$$\hat{x}_{k|N} = \hat{x}_{k|k} + C_k \left(\hat{x}_{k+1|N} - \hat{x}_{k+1|k} \right) \quad (42)$$

mentre la covarianza associata viene aggiornata secondo

$$P_{k|N} = P_{k|k} + C_k \left(P_{k+1|N} - P_{k+1|k} \right) C_k^T. \quad (43)$$

Grazie all'utilizzo delle informazioni future, lo smoother RTS è in grado di ridurre il rumore residuo presente nelle stime e migliorare la ricostruzione delle variabili non direttamente osservabili. Il codice MATLAB che implementa lo smoother RTS è riportato nel listato seguente:

```

1 %% Inizializzazione RTS
2
3 x_hat_smoothed = zeros(N, nx);
4 P_smoothed = zeros(nx, nx, N);
5
6 % All'ultimo istante, stima smoothed = stima filtrata EKF
7 x_hat_smoothed(N, :) = x_corr(N, :);
8 P_smoothed(:, :, N) = P_corr_3D(:, :, N);
9
10 %% Esecuzione backward dello smoother RTS
11
12 for k = N-1:-1:1
13
14     % Covarianza corretta al tempo k
15     Pkk = P_corr_3D(:, :, k);
16
17     % Grandezze predette al tempo k+1
18     Pkp1_pred = P_pred_3D(:, :, k+1);
19     Fkp1 = F_3D(:, :, k+1);
20
21     % Guadagno RTS
22     Ck = Pkk * Fkp1' / Pkp1_pred;
23
24
25     % Calcolo della differenza vettoriale tra stato futuro smoothed e futuro
    predetto
26     diff_stato = x_hat_smoothed(k+1, :) - x_pred(k+1, :);
27
28     % si applica il wrap solo agli errori degli angoli (alpha e beta)
29     diff_stato(1) = mod(diff_stato(1) + pi, 2*pi) - pi;
30     diff_stato(3) = mod(diff_stato(3) + pi, 2*pi) - pi;
31
32     x_hat_smoothed(k, :) = (x_corr(k, :) + Ck * diff_stato)';
33
34     % Covarianza smoothed
35     P_smoothed(:, :, k) = Pkk + ...
36         Ck * (P_smoothed(:, :, k+1) - Pkp1_pred) * Ck';

```

```

37
38     % Simmetrizzazione numerica
39     P_smoothed(:, :, k) = 0.5 * (P_smoothed(:, :, k) + P_smoothed(:, :, k)');
40 end

```

Listing 5: Smoother RTS

6 Analisi dei risultati

Per valutare le prestazioni degli algoritmi sviluppati sono stati analizzati gli andamenti temporali degli stati stimati, delle matrici di covarianza e delle innovazioni. Le simulazioni sono state effettuate utilizzando traiettorie ottenute mediante ingressi sinusoidali applicati ai due motori, in modo da eccitare simultaneamente le dinamiche di pitch e yaw. L'errore di stima è stato definito come

$$e_x(t) = x_{vero}(t) - \hat{x}(t) \quad (44)$$

e valutato separatamente per ciascuna componente dello stato. Oltre all'errore di stima sono state analizzate le innovazioni prodotte dai filtri, verificandone la consistenza statistica mediante test di bianchezza basati sull'autocorrelazione campionaria normalizzata con intervallo di confidenza al 95 % e considerando 50 ritardi. I risultati dei test di bianchezza sulle innovazioni sono riportati per l'EKF nelle Figure 21, 22 e per l'UKF nelle Figure 24, 25. Per il magnetometro, essendo l'innovazione composta da 2 componenti che rappresentano lo stesso vettore di campo e che sono quindi correlate tra loro, è stata testata un'innovazione angolare equivalente ottenuta proiettando le 2 componenti lungo la direzione sensibile all'angolo di imbardata, mediante la formula

$$\frac{\sin(\hat{\beta})e_{m1} + \cos(\hat{\beta})e_{m2}}{M_0} \quad (45)$$

L'obiettivo di tale analisi consiste nel verificare che le informazioni contenute nelle misure siano state correttamente sfruttate dal filtro e che non permangano dinamiche sistematiche non modellate.

6.1 Prestazioni dell'EKF

Le simulazioni effettuate mostrano che il filtro di Kalman esteso è in grado di ricostruire correttamente l'evoluzione delle variabili di stato dell'elicottero. Le stime ottenute risultano generalmente sovrapposte alle traiettorie reali e gli errori rimangono confinati entro intervalli compatibili con l'incertezza introdotta dai rumori di processo e misura come visibile in Figura 20, in cui si nota che gli errori angolari sono limitati ad un ordine di grandezza di 10^{-3} rad e quelli sul rate angolare a $10^{-1} \frac{\text{rad}}{\text{s}}$. Le componenti associate al pitch presentano le migliori prestazioni, grazie alla presenza di una misura accelerometrica direttamente correlata all'angolo (α).

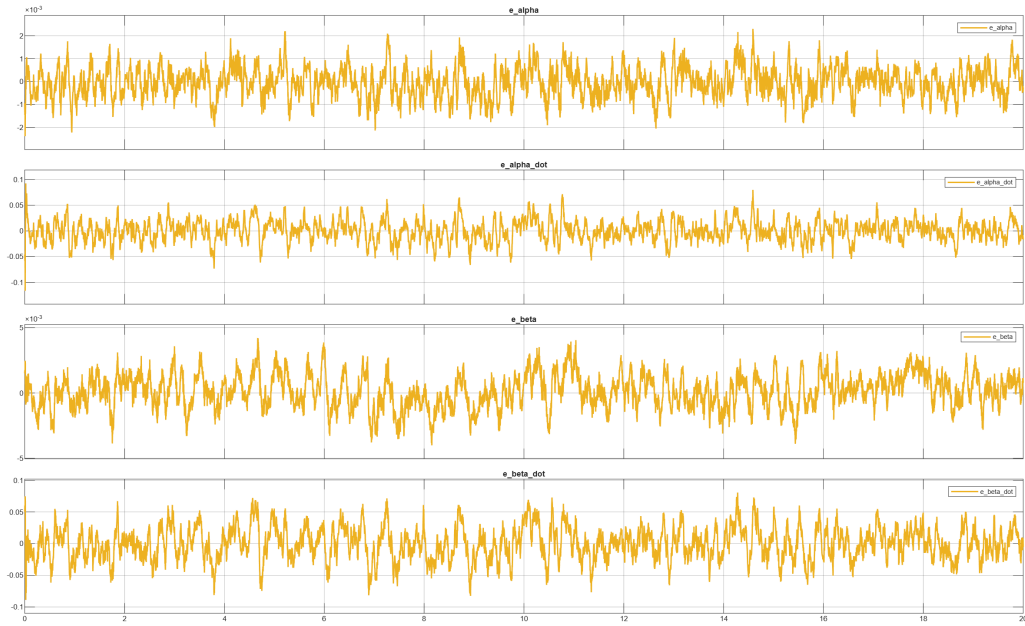


Figura 20: Errori di stima per l'EKF

L'errore sui rate angolari è maggiore rispetto a quello sugli angoli di beccheggio ed imbardata, in quanto le prime sono grandezze vengono ricostruite attraverso il modello dinamico e le correlazioni contenute nella covarianza, risultando più sensibili a rumore di processo ed agli eventuali errori di modellazione.

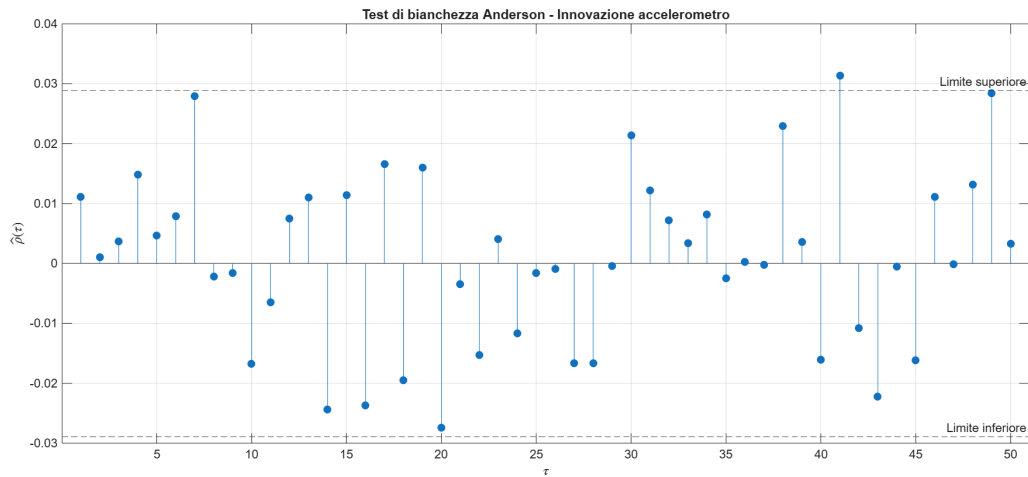


Figura 21: Test di bianchezza per l'innovazione relativa all'accelerometro (EKF).

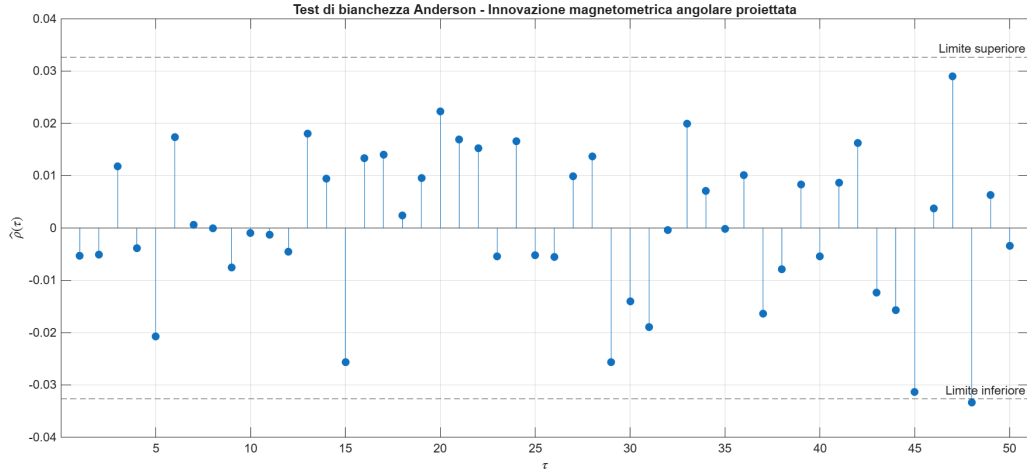


Figura 22: Test di bianchezza per l'innovazione relativa al magnetometro (EKF)

6.2 Prestazioni dell'UKF

Il filtro di Kalman Unscented ha mostrato prestazioni complessivamente comparabili rispetto all'EKF. Le prestazioni in termini di errori di stima dell'UKF sono analizzabili a partire dai grafici dell'errore di stima riportati in Figura 23.

L'UKF presenta inoltre una maggiore robustezza nei confronti delle approssimazioni numeriche associate alle matrici Jacobiane, completamente assenti nell'implementazione Unscented. Dal punto di vista computazionale, tuttavia, il costo richiesto dall'UKF risulta superiore rispetto a quello dell'EKF, poiché ad ogni iterazione è necessario generare e propagare un insieme completo di Sigma Points. Nonostante tale incremento di complessità, le prestazioni ottenute giustificano l'utilizzo dell'approccio Unscented in applicazioni caratterizzate da elevate non linearità.

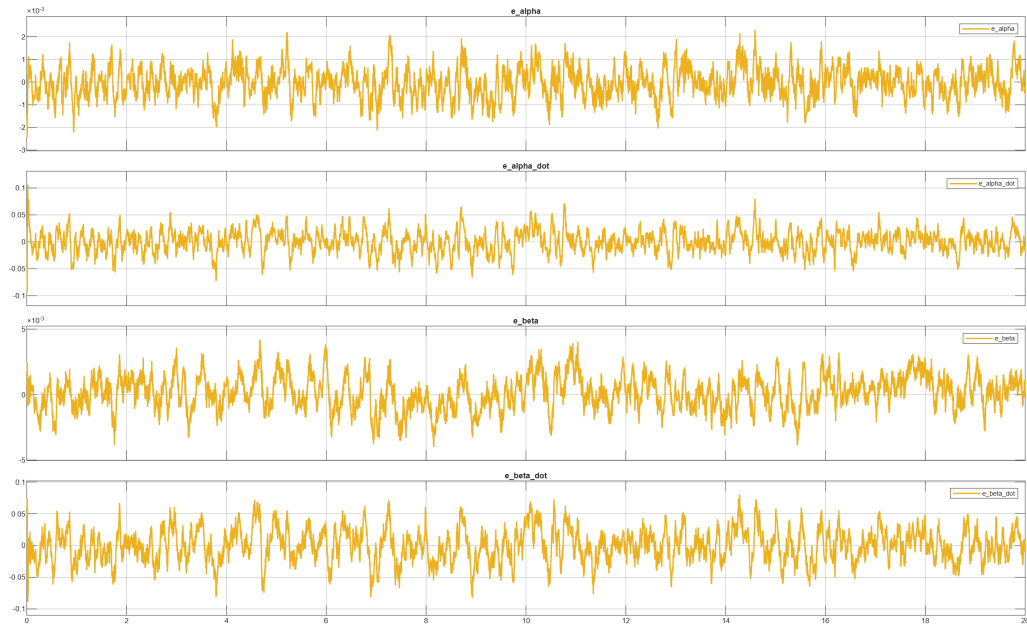


Figura 23: Errori di stima per l'UKF

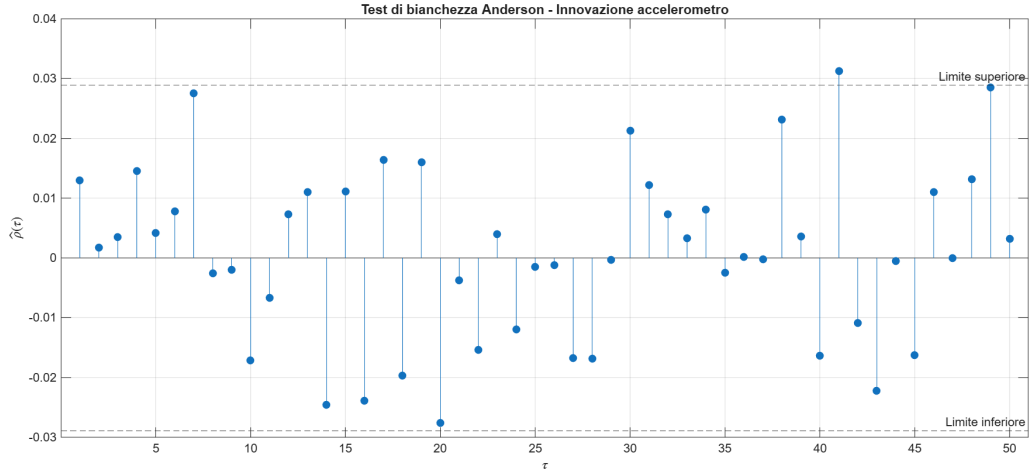


Figura 24: Test di bianchezza per l'innovazione relativa all'accelerometro (UKF).

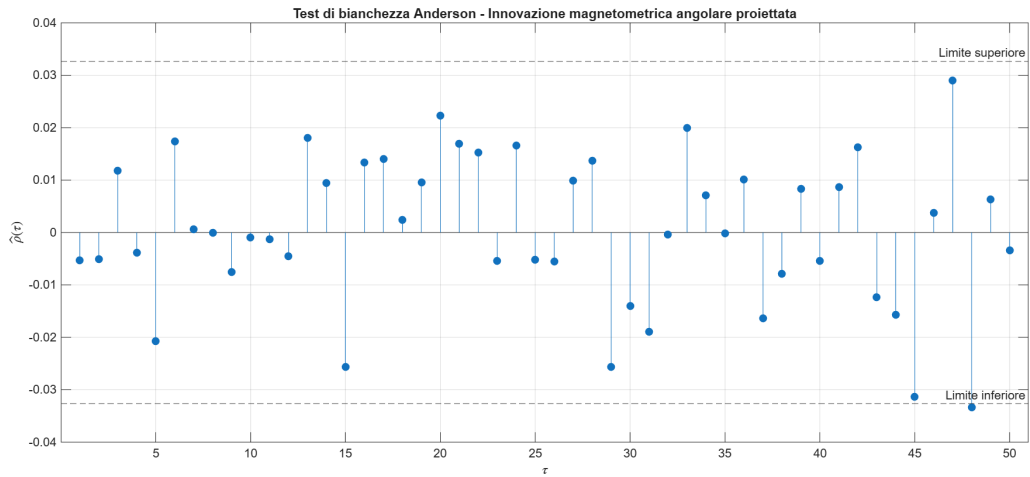


Figura 25: Test di bianchezza per l'innovazione relativa al magnetometro (UKF)

6.3 Confronto tra EKF e UKF

Il confronto tra i due algoritmi evidenzia differenze principalmente riconducibili al metodo utilizzato per la propagazione dell'incertezza. L'EKF utilizza una linearizzazione locale ottenuta tramite sviluppo di Taylor al primo ordine. Tale approssimazione risulta generalmente adeguata quando il sistema opera in prossimità di condizioni quasi lineari. L'UKF, al contrario, evita completamente il processo di linearizzazione e sfrutta la Trasformata Unscented per propagare direttamente media e covarianza attraverso le equazioni non lineari. Dal punto di vista dell'accuratezza, l'UKF fornisce generalmente stime leggermente migliori delle variabili maggiormente influenzate dalle non linearità del modello. L'EKF presenta invece una minore complessità computazionale e una struttura più semplice da implementare e validare. Nel contesto del presente progetto entrambi i filtri hanno mostrato prestazioni soddisfacenti, consentendo una corretta ricostruzione dello stato dell'elicottero.

6.4 Confronto tra EKF e RTS

L'analisi comparativa tra EKF e RTS evidenzia come le stime prodotte dai due algoritmi risultino generalmente molto simili. Lo smoother RTS introduce un miglioramento principalmente nelle componenti caratterizzate da maggiore rumore e nelle derivate angolari, che risultano più regolari rispetto alle corrispondenti stime filtrate come visibile nelle Figure 26, 28. L'andamento invece delle covarianze dell'errore di stima, visibile nelle Figure 27 e 29 mostra come lo smoother performi in modo migliore in presenza di outlier in quanto la covarianza nell'EKF in corrispondenza dell'EKF aumenta drasticamente, mentre nel caso del RTS rimane pressochè costante. Ciò è dovuto al fatto che nel caso in cui l'EKF salti il passo di correzione, la stima risulta una predizione pura, pertanto meno affidabile rispetto alla regolarizzazione mediante i valori futuri effettuata dallo smoother.

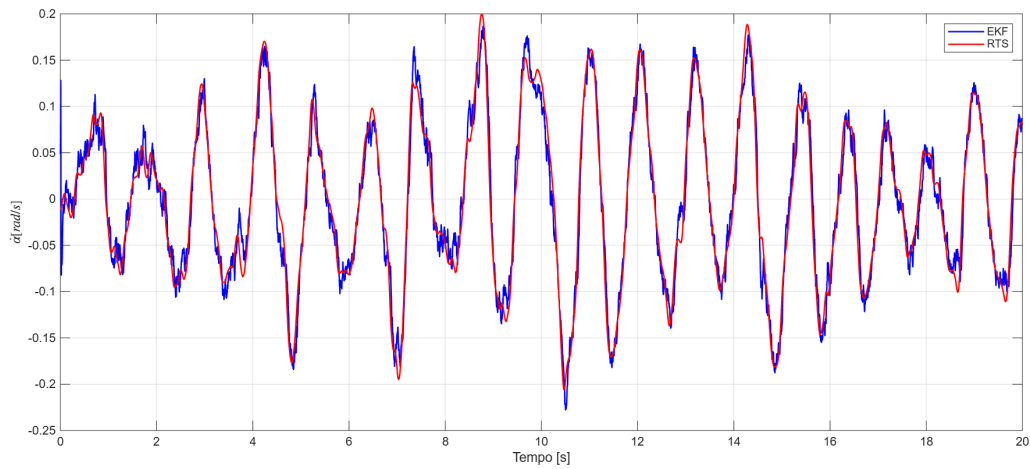


Figura 26: Confronto tra stima EKF ed RTS per il rate angolare $\dot{\alpha}$

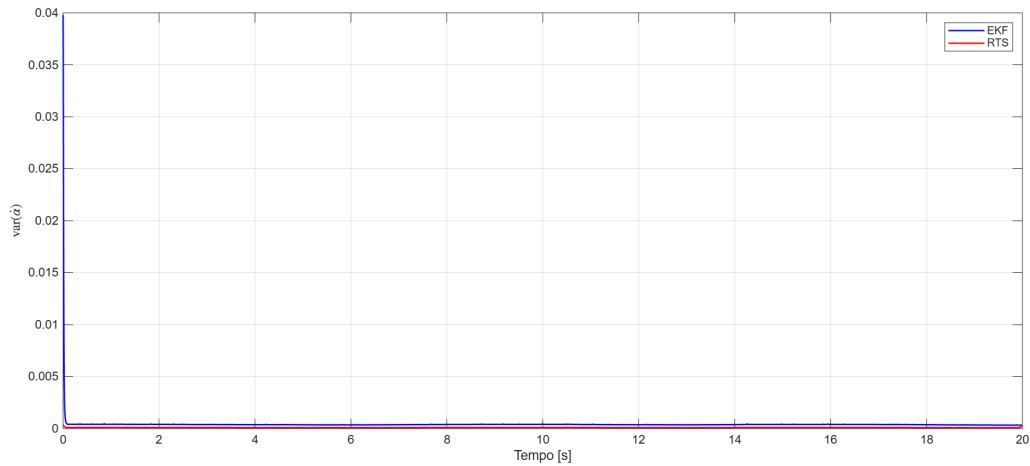


Figura 27: Confronto tra stima EKF ed RTS per la varianza del rate angolare $\dot{\alpha}$

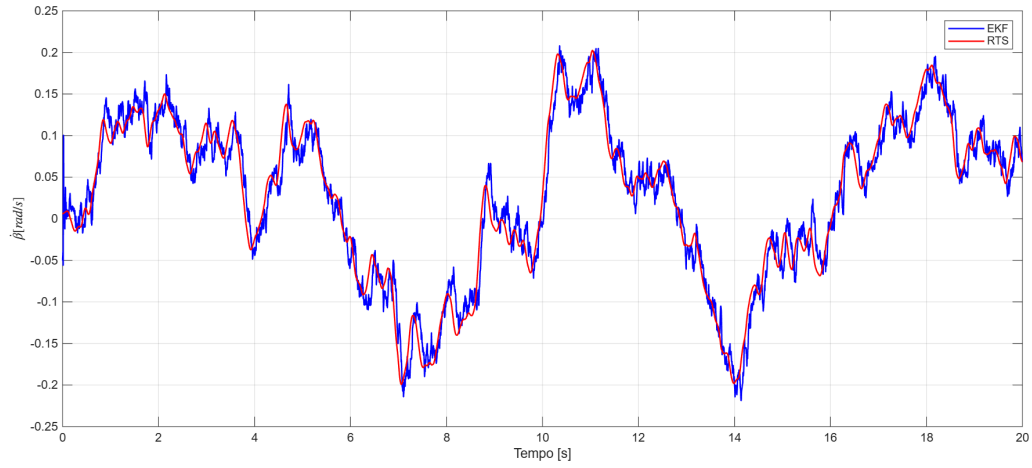


Figura 28: Confronto tra stima EKF ed RTS per il rate angolare $\dot{\beta}$

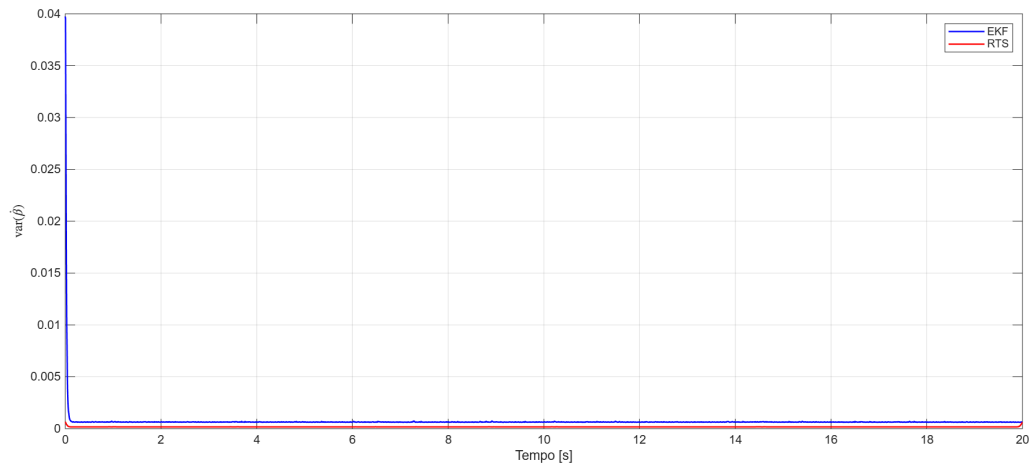


Figura 29: Confronto tra stima EKF ed RTS per la varianza del rate angolare $\dot{\beta}$

Tale comportamento è verificabile anche per le misure angolari riportate di seguito nelle Figure 30, 31, 32 ed 33.

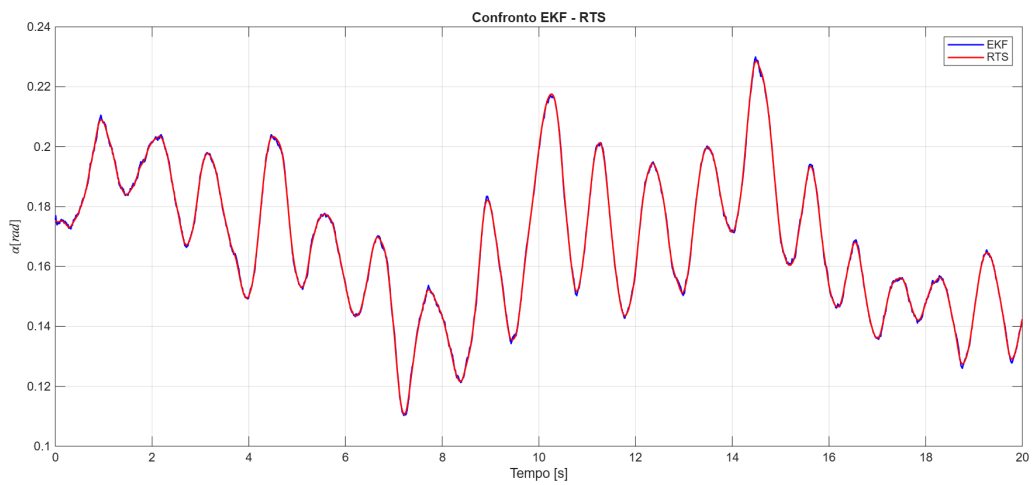


Figura 30: Confronto tra stima EKF ed RTS per l'angolo α

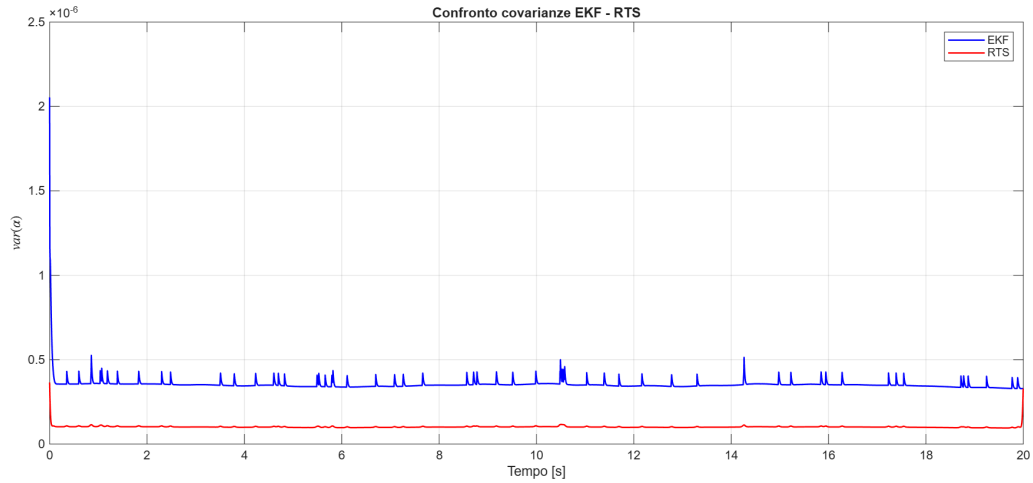


Figura 31: Confronto tra stima EKF ed RTS per la varianza dell'angolo α

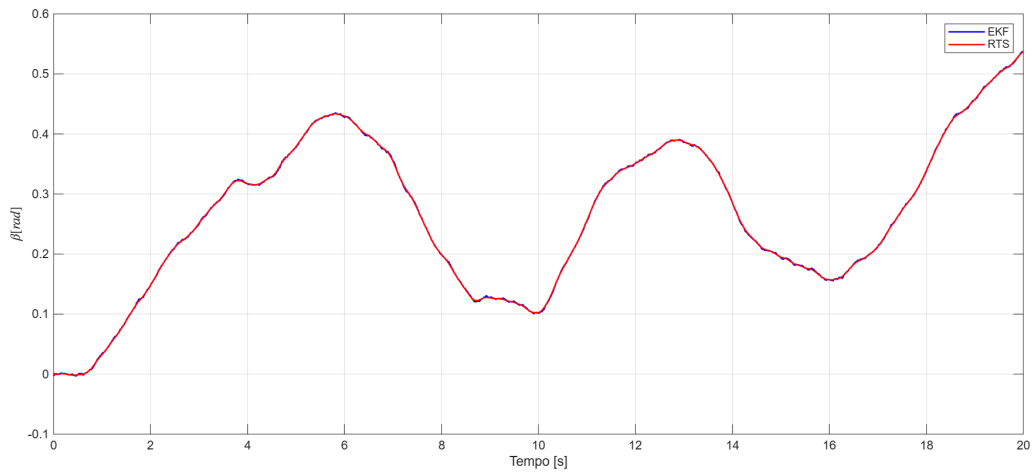


Figura 32: Confronto tra stima EKF ed RTS per l'angolo β

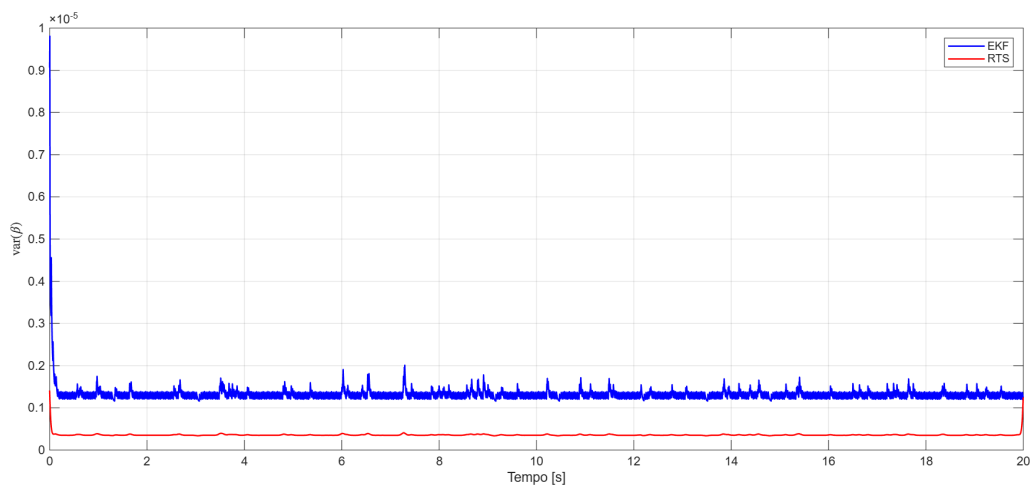


Figura 33: Confronto tra stima EKF ed RTS per la varianza dell'angolo β

I vantaggi dell'RTS diventano particolarmente evidenti nelle applicazioni offline, quali analisi sperimentali, identificazione parametrica e post-processing di dati acquisiti.

Viceversa, nelle applicazioni in tempo reale il filtro non può essere utilizzato poiché richiede la disponibilità dell'intera sequenza di osservazioni. I risultati ottenuti mostrano che l'EKF fornisce già una stima sufficientemente accurata per applicazioni online, mentre l'RTS consente di ottenere una ricostruzione più regolare e statisticamente consistente delle traiettorie.

7 Conclusioni

Nel presente lavoro sono stati sviluppati e validati differenti algoritmi di stima applicati ad un modello non lineare di elicottero a due gradi di libertà. Partendo dalla realizzazione del modello dinamico e della relativa sensoristica virtuale, sono stati implementati un filtro di Kalman esteso, un filtro di Kalman Unscented ed uno smoother di Rauch-Tung-Striebel. L'analisi dei risultati ha mostrato come tutti gli algoritmi siano in grado di ricostruire correttamente lo stato del sistema anche in presenza di rumori di processo, rumori di misura e misure spurie. L'EKF si è dimostrato una soluzione efficace e relativamente semplice da implementare, a fronte di un minore costo computazionale rispetto all'UKF. Quest'ultimo si sarebbe dimostrato più performante nel caso in cui le non linearità del modello fossero state particolarmente accentuate. Lo smoother RTS ha infine consentito di migliorare ulteriormente la qualità delle stime attraverso un'elaborazione offline delle traiettorie. Particolare attenzione è stata dedicata alla robustezza numerica dell'implementazione, introducendo tecniche di validazione statistica mediante distanza di Mahalanobis, gestione multi-rate delle misure e procedure di stabilizzazione numerica basate sulla decomposizione ai valori singolari. I risultati ottenuti confermano la validità delle metodologie adottate e mostrano come gli strumenti di sensor fusion basati sul filtro di Kalman rappresentino una soluzione efficace per la stima dello stato di sistemi caratterizzati da dinamiche non lineari.

Appendice

A Confronti delle stime con andamenti reali

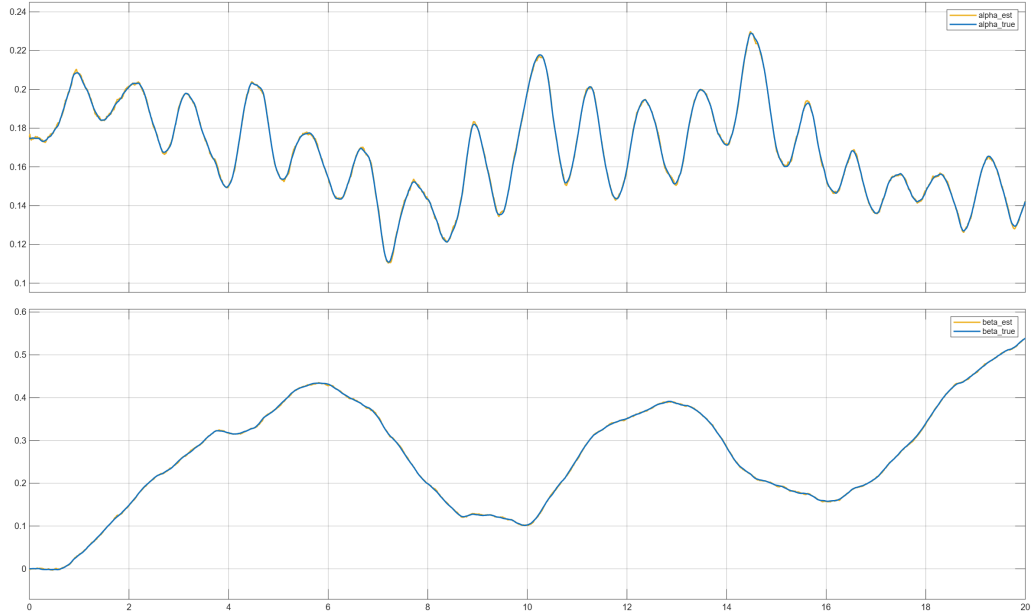


Figura 34: Confronto tra angoli di beccheggio ed imbardata reali e stimati con EKF

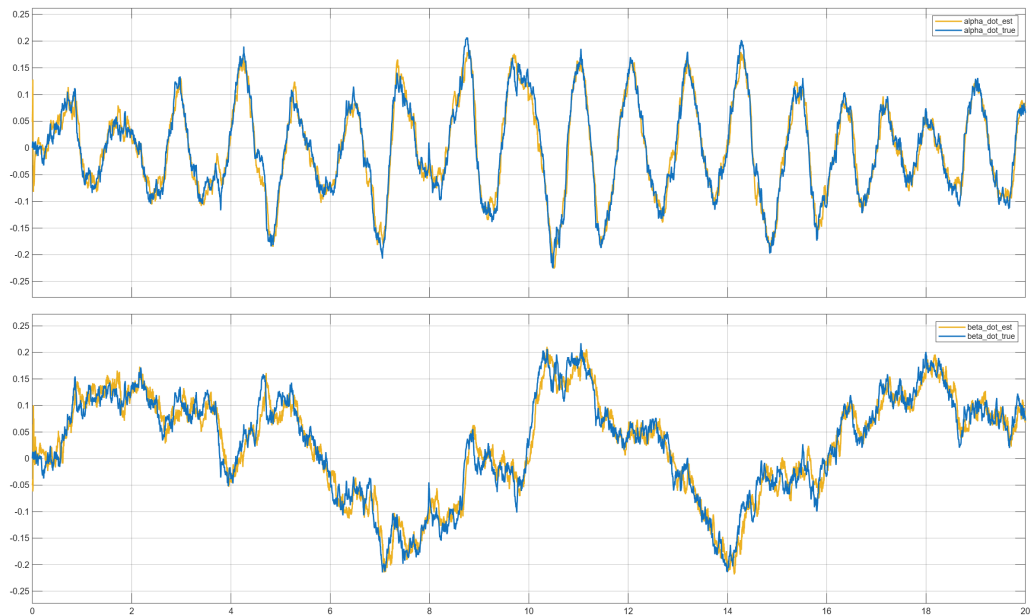


Figura 35: Confronto tra rate angolari di beccheggio ed imbardata reali e stimati con EKF

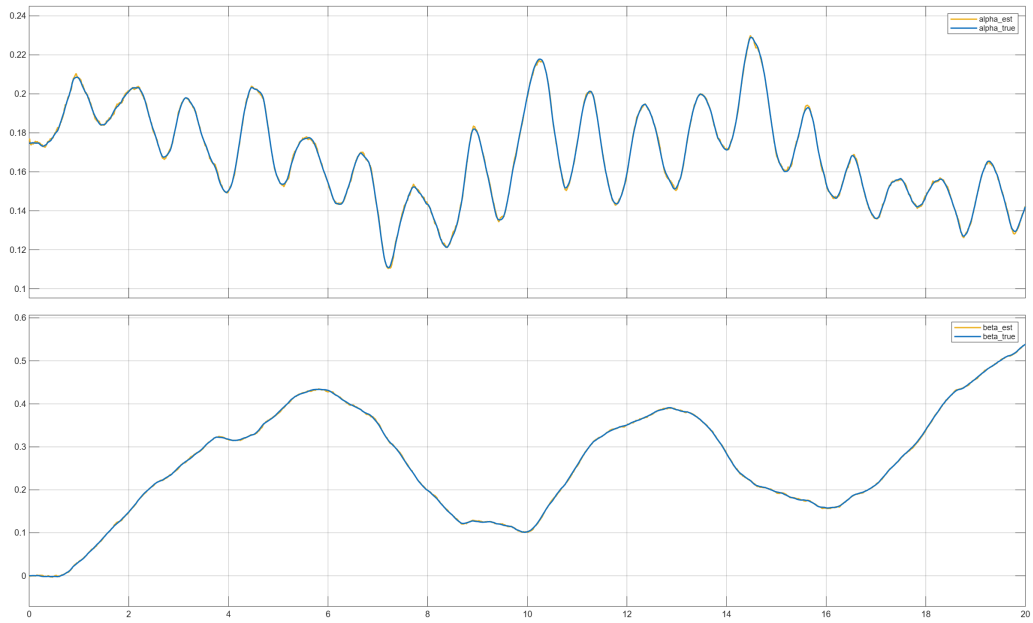


Figura 36: Confronto tra angoli di beccheggio ed imbardata reali e stimati con UKF

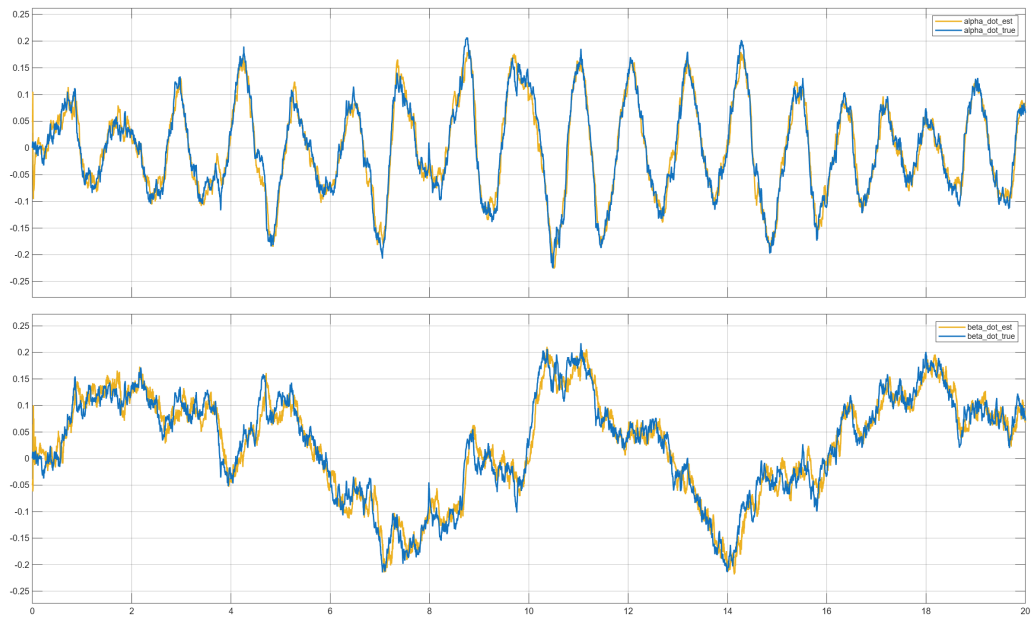


Figura 37: Confronto tra rate angolari di beccheggio ed imbardata reali e stimati con UKF

B Script MATLAB di inizializzazione

```
1 %% FILE DI INIZIALIZZAZIONE PER STIMA ELICOTTERO 2DOF
2 % Autori: Edoardo Di Scalzo, Luigi Realacci, Francesco Bonciani
3 clear
4 close all
5 clc
6
7 %-----
8 % Parametri del modello
9 %-----
10
11 % Parametri Inerziali e di massa
12 params.m = 0.2; % [kg] Massa del sistema
13 params.J_alpha = 0.012; % [kg*m^2] Mom. di Inerzia rispetto all'asse di pitch
14 params.J_y = 2.3e-4; % [kg*m^2] Componente di Inerzia Trasversale
15 params.J_z = 36.4e-4; % [kg*m^2] Componente di Inerzia Verticale
16 params.I_b = 2.3e-4; % [kg*m^2] Componente di Inerzia della base
17
18
19 % Parametri geometrici e meccanici
20 params.g = 9.81; % [m/s^2] Accelerazione di gravita'
21 params.l = 0.2; % [m] Distanza tra perno di rotazione e applicazione
    forze rotori
22 params.c_alpha = 0.01; % [N*m*s/rad] Coefficiente di attrito viscoso asse di
    pitch
23 params.c_beta = 0.01; % [N*m*s/rad] Coefficiente di attrito viscoso asse di
    yaw
24
25 % Parametri Aerodinamici
26 params.epsilon_p = 0.1; % Coefficiente di cross-thrust del rotore di coda
    sull'asse del pitch
27 params.epsilon_y = 0.1; % Coefficiente di cross-thrust del rotore principale
    sull'asse dello yaw
28
29 % Sample time
30 params.st = 0.00435; % Pari al sample time del sensore piu veloce (
    accelerometro)
31
32 Tf = 20; % Tempo di simulazione
33 t = (0:params.st:Tf)';
34
35 %-----
36 % Condizioni Iniziali
37 %-----
38
39 alpha_0 = deg2rad(10); % [rad] Angolo di pitch iniziale
40 alpha_dot_0 = 0; % [rad/s] Derivata angolo di pitch iniziale
41 beta_0 = 0; % [rad] Angolo di yaw iniziale
42 beta_dot_0 = 0; % [rad/s] Derivata angolo di yaw iniziale
43
44
45 % Vettori condizioni iniziali (Vettori colonna!)
```

```

46 q_0 = [alpha_0 beta_0]'; % Vettore condizioi iniziali per alpha e
    beta
47 q_dot_0 = [alpha_dot_0 beta_dot_0]'; % Vettore condizioni iniziali per
    alpha_dot e beta_dot
48
49 %-----
50 % Input del sistema (forze F1 ed F2)
51 %-----
52 F1 = 0.341 + 0.04 * sin(0.6 * t) + 0.02 * sin(1.4 * t); % [N]
    Forza del rotore principale
53 F2 = -0.006 + 0.010 * sin(0.8 * t + 0.3) + 0.006 * sin(1.7 * t); % [N]
    Forza del rotore di coda
54 %F1 = 0 * t;
55 %F2 = 0 * t;
56 u = [t, F1, F2];
57
58 %% SENSORI: Accelerometro e Magnetometro (Vectornav vn-100)
59 %-----
60 % Parametri Sensori
61 %-----
62 seme_rumore = 12345; % Seme per rumore additivo dei sensori
63
64 % Accelerometro
65 sigma_acc = 0.0137; % [m/s^2] Dev. Std. Accelerometro
66 st_acc = 0.00435; % Sample time Accelerometro (230 Hz)
67
68 % Magnetometro
69 M_0 = 46.5; % [uT] Intensita' campo magnetico locale
70 sigma_magn = 0.14; % [muT] Dev. Std. Magnetometro
71 st_magn = 0.005; % Sample time Magnetometro (200 Hz)
72
73
74 %% PARAMETRI GESTIONE OUTLIER E MAHALANOBIS
75
76 % Attiva/Disattiva Outlyer (1 attivi, 0 disattivati)
77 Outlier_flag = 0;
78
79 % Flag per attivare/Disattivare Distanza di Mahalanobis
80 MAHALANOBIS = false;
81
82 % Treshold per distanza di mahalanobis
83 m_treshold = 3;
84
85 %% Parametri EKF
86
87 %-----
88 % Condizioni iniziali
89 %-----
90 P_0_alpha = deg2rad(5)^2;
91 P_0_alpha_dot = 0.20^2;
92 P_0_beta = deg2rad(8)^2;
93 P_0_beta_dot = 0.20^2;
94

```

```

95 x_0 = [(deg2rad(10) + deg2rad(3)) 0.05 deg2rad(5) -0.05]';           % Stato
    iniziale
96
97 % Matrice delle covarianze iniziale
98 P0 = diag([P_0_alpha P_0_alpha_dot P_0_beta P_0_beta_dot]);
99
100
101 % Covarianza rumore di processo
102 % params.sigma_alpha = 1e-5;           % Dev. Std. alpha
103 % params.sigma_alpha_dot = 1e-4;       % Dev. Std. alpha_dot
104 % params.sigma_beta = 1e-5;           % Dev. Std. beta
105 % params.sigma_beta_dot = 1e-4;       % Dev. Std. beta_dot
106 params.sigma_F = 1.8 * 0.05;          % [N] Dev.Std. Ingresso
107
108 params.R_acc = sigma_acc^2;
109 params.R_magn = sigma_magn^2;
110
111 %% Parametri UKF
112
113 % SUKF (Filtro di Kalman Unscented Scalato)
114 params.Alpha_UKF = 1e-2;              % Parametro Alpha UKF (Dispersione sigma-points)
115 params.Beta_UKF = 2;                  % Parametro Beta UKF (Ottimizza termini di ordine
    superiore)
116 params.Kappa_UKF = 0;                 % Parametro Kappa UKF (Parametro di scaling
    secondario)
117
118 params.n_state = length(x_0);          % Dimesnione stato
119 params.n_input = 2;                   % Dimensione input
120
121 % Caso con rumore di attuazione
122 params.n = params.n_state + params.n_input;
123 params.N_sigma_points = 2 * params.n + 1; % Numero di sigma points (2*6 + 1
    = 13)
124
125 % Parametro Lambda UKF
126 % Per fase di predizione
127 params.lambda = params.Alpha_UKF^2 * (params.n + params.Kappa_UKF) - params.n;
128 % Per fase di correzione
129 params.lambda_c = params.Alpha_UKF^2 * (params.n_state + params.Kappa_UKF) -
    params.n_state;
130
131 % Assegnamento pesi per media e covarianza
132 n = params.n;
133 params.Wm = zeros(2*n + 1, 1);        % Pesi media
134 params.Wc = zeros(2*n + 1, 1);        % Pesi covarianza
135 % Parte di predizione
136 for i = -n:n
137     idx = i + n + 1;
138     if i == 0
139         % Peso centrale
140         params.Wm(idx) = params.lambda / (n + params.lambda);
141         params.Wc(idx) = (params.lambda / (n + params.lambda)) + 1 - params.
            Alpha_UKF^2 + params.Beta_UKF;
142     else

```

```

143         % Pesi restanti
144         params.Wm(idx) = 1 / (2 * (n + params.lambda));
145         params.Wc(idx) = 1 / (2 * (n + params.lambda));
146     end
147 end
148
149 % Parte di correzione (i sigma points sono 9 poiche' si sfrutta rumore
150 % additivo)
151 n_state = params.n_state;
152 params.Wm_c = zeros(2*n_state + 1, 1);           % Pesi media
153 params.Wc_c = zeros(2*n_state + 1, 1);           % Pesi covarianza
154 for i = -n_state:n_state
155     idx = i + n_state + 1;
156     if i == 0
157         % Peso centrale
158         params.Wm_c(idx) = params.lambda_c / (n_state + params.lambda_c);
159         params.Wc_c(idx) = (params.lambda_c / (n_state + params.lambda_c)) + 1 -
            params.Alpha_UKF^2 + params.Beta_UKF;
160     else
161         % Pesi restanti
162         params.Wm_c(idx) = 1 / (2 * (n_state + params.lambda_c));
163         params.Wc_c(idx) = 1 / (2 * (n_state + params.lambda_c));
164     end
165 end
166
167 % Chiamata al simulatore su Simulink
168 out = sim('Elicottero2DOF_Sim.mdl');

```

Listing 6: Funzione di inizializzazione